

Module I: A simple Java Application, a simple Java Applet , Brief History of Java, Special Features of Java, Data Type & Operators in Java, Arrays, Objects, the Assignment Statement, Arithmetic Operators, Relational and Logical Operators in Java, control Structures, The Java Class, Constructor, Finalizers, Classes inside classes: composition

OVERVIEW OF JAVA

Java was conceived by James Gosling, Patrick Naughton, Chris Warth, Ed Frank, and Mike Sheridan at Sun Microsystems, Inc. in 1991. It took 18 months to develop the first working version. This language was initially called “Oak” but was renamed “Java” in 1994 as the Web emerged.

HotJava

HotJava is a Web browser that is written in Java. HotJava is a Java-enabled browser. This means that HotJava can execute Java applets contained on Web pages. In order to accomplish this, HotJava calls the Java runtime system. The Netscape 2.0 browser, like HotJava, is also Java enabled. It contains a copy of the Java runtime system embedded within it.

Basic features

The Java team has summed up the **basic features** of Java with the following list of buzzwords :

- Simple

Java was designed to be easy for the professional programmer to learn and use effectively. Assuming that you have some programming experience, you will not find Java hard to master. If you already understand the basic concepts of object-oriented programming, learning Java will be even easier. Best of all, if you are an experienced C++ programmer, moving to Java will require very little effort. Because Java inherits the C/C++ syntax and many of the object-oriented features of C++, most programmers have little trouble learning Java.

- Object-oriented

Java is a true object-oriented language. Many of Java’s object-oriented concepts are inherited from C++, the language on which it is based, but it borrows many concepts from other object-oriented languages as well. Like most object-oriented programming languages, Java includes a set of class libraries that provide basic data types, system input and output capabilities, and other utility functions. These basic classes are part of

the Java development kit, which also has classes to support networking, common Internet protocols, and user interface toolkit functions.

- Robust

Java is a robust language. The multiplatform environment of the Web places extraordinary demands on a program, because the program must execute reliably in a variety of systems. Thus, the ability to create robust programs was given a high priority in the design of Java. To gain reliability, Java restricts you in a few key areas, to force you to find your mistakes early in program development. At the same time, Java frees you from having to worry about many of the most common causes of programming errors. Because Java is a strictly typed language, it checks your code at compile time. However, it also checks your code at run time.

- Secure

Prior to Java, most users did not download executable programs frequently from Internet, and those who did scanned them for viruses prior to execution. Even so, most users still worried about the possibility of infecting their systems with a virus. In addition to viruses, another type of malicious program exists that must be guarded against. This type of program can gather private information, such as credit card numbers, bank account balances, and passwords, by searching the contents of your computer's local file system. Java answers both of these concerns by providing a "firewall" between a networked application and your computer. When you use a Java-compatible Web browser you can safely download Java applets without fear of viral infection or malicious intent. Java achieves this protection by confining a Java program to the Java execution environment and not allowing it access to other parts of the computer.

- Multithreaded

Java was designed to meet the real-world requirement of creating interactive, networked programs. To accomplish this, Java supports multithreaded programming, which allows you to write programs that do many things simultaneously. The Java run-time system comes with an elegant yet sophisticated solution for multiprocessor synchronization that enables you to construct smoothly running interactive systems.

- Architecture-neutral

A central issue for the Java designers was that of code longevity and portability. One of the main problems facing programmers is that no guarantee exists that if you write a program today, it will run tomorrow— even on the same machine. Operating system

upgrades, processor upgrades, and changes in core system resources can all combine to make a program malfunction. The Java designers made several hard decisions in the Java language and the Java Virtual Machine in an attempt to alter this situation. Their goal was “write once; run anywhere, any time, forever.” To a great extent, this goal was accomplished.

- Portable

In addition to being architecture-neutral, Java code is also portable. It was an important design goal of Java that it be portable so that as new architectures (due to hardware, operating system, or both) are Java and the runtime environment is written in POSIX-compliant C.

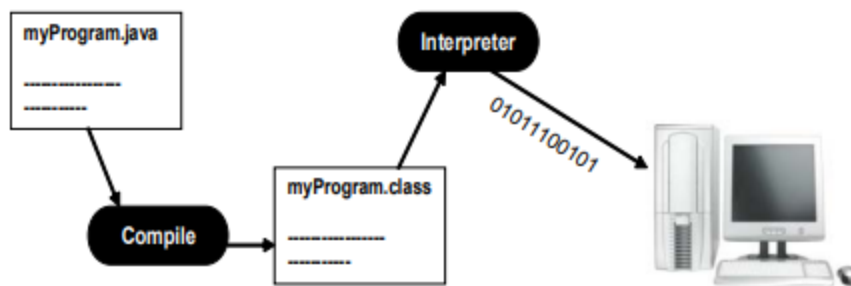
- Distributed

Java is designed for the distributed environment of the Internet, because it handles TCP/IP protocols. In fact, accessing a resource using a URL is not much different from accessing a file. The original version of Java (Oak) included features for intra-address space messaging. This allowed objects on two different computers to execute procedures remotely. Java has recently revived these interfaces in a package called Remote Method Invocation (RMI). This feature brings an unparalleled level of abstraction to client/server programming.

- Dynamic

Java programs carry with them substantial amounts of run-time type information that is used to verify and resolve accesses to objects at run time. This makes it possible to dynamically link code in a safe and expedient manner. This is crucial to the robustness of the applet environment, in which small fragments of bytecode may be dynamically updated on a running system.

The Java development environment has two parts: a Java compiler and a Java interpreter. In the Java programming language, all source code is first written in plain text files ending with the .java extension. Those source files are then compiled into .class files by the javac compiler. With the compiler, first you translate a program into an inter-mediate code called Java bytecodes. Bytecodes are not machine instructions and therefore, in the second stage, Java interpreter generates machine code that can be directly executed by the machine that is running the Java program. The interpreter parses and runs each Java bytecode instruction on the computer. Compilation happens just once; interpretation occurs each time the program is executed.



Java Virtual Machine (JVM) Java compiler produces an intermediate code known as bytecode for a machine that does not exist. This machine is called the JVM and it exists only inside the computer memory. The bytecodes are also known as virtual machine code which are not the actual machine code. The actual machine codes are generated by the Java interpreter only.

JAVA ENVIRONMENT

Java environment includes a large number of development tools and hundreds of classes and methods. The development tools are part of the system known as Java development Kit (JDK) and the classes and methods are part of the Java Standard Library (JSL), also known as the Application Programming Interface (API). Java development Kit the Java development Kit comes with a collection of tools that are used for development and running Java programs. Some of them are :

java The loader for Java applications. This tool is an interpreter and can interpret the class files generated by the javac compiler.

javac The compiler, which converts source code into Java bytecode

jar The archiver, which packages related class libraries into a single JAR file.

javadoc The documentation generator, which automatically generates documentation from source code comments

jdb The Java debugger

jps The process status tool, which displays process information for current Java processes

javap The class file disassembler

appletviewer This tool can be used to run and debug Java applets without a web browser.

javah The C header and stub generator, used to write native methods

An **application programming interface** (API) is an interface implemented by a software program to enable interaction with other software, similar to the way a user interface facilitates interaction between humans and computers. Java APIs include hundreds of classes and methods grouped into several functional packages. Most commonly used packages are :

- Language support package
- Utility package
 - Input/Output Package
- Networking Package
 - AWT(Abstract Window Tool Kit) Package
 - Applet Package

two types of Java Programs :

- Stand alone applications
- Java applets

Stand alone applications are java programs that can carry out certain tasks on local computer. Java applets are small programs that are used to developed Internet applications. Java applets can be downloaded from a Web server and run on your computer by a Java-compatible Web browser, such as Netscape Navigator or Microsoft Internet Explorer.

- Create a source file. A source file contains text, written in the Java programming language, that you and other programmers can understand. You can use any text editor (e.g. Notepad) to create and to edit source files.
- Compile the source file into a bytecode file. The compiler takes your source file and translates the text into instructions that the Java VM can understand. The compiler converts these instructions into a bytecode file.
- Run the program contained in the bytecode file. The Java interpreter installed on your computer implements the Java VM. This interpreter takes your bytecode file and carries out the instructions by translating them into instructions that your computer can understand.

simple Java program using Notepad:

```
class Test
{
public static void main(String[] args)
{
System.out.println("I am A Simple Program!");
}
```

```
}  
}
```

After typing the program, save it in a folder by giving the file name same as the class name (here it is Test) with the extension .java.

for compiling the source file i.e. Test.java enter the following command at Command Prompt - javac Test.java

To run your program by typing the following command -
java Test

Class Declaration

A class is the basic building block of an object-oriented language, such as the Java programming language. The above Java program consists of a main class, named Test, which contains a main function, named main(). The following bold text begins the class definition block for the application :

```
class Test  
{  
    public static void main(String[] args)  
    {  
        System.out.println("I am A Simple Program!");  
    }  
}
```

The main() Method or Fuction

The following is the definition of the main method :

```
class Test  
{  
    public static void main(String[] args)  
    {  
        System.out.println("I am A Simple Program!");  
    }  
}
```

Every Java program must include the main() function declared like this:

public static void main(String[] args)

Conceptually, this is similar to the main() function in C / C++ and it's the entry point for your application and will subsequently invoke all the other methods required by your program. The main method declaration starts with three modifiers whose meanings are given below :

public : means allows any class to call the main method

static : means that the main method is associated with the Test class as a whole instead of operating on an instance or object of the class

void : indicates that the main method does not return a value As you can see from the declaration of the main() function,

public static void main(String[] args)

it accepts a single argument i.e. String[] means an array of elements of type String. The name of this array is args (for “arguments”). This array is the mechanism through which the Java Virtual Machine passes information to your application.

The Output Line

The only executable statement in the program is

`System.out.println("I am A Simple Program!");`

This is similar to the “printf()” statement of C or “cout <<” of C++.

The println method is a member of the out object, which is a static

data member of System class. This line prints the string - I am A Simple Program!

to the screen. The method println always appends a newline character to the end of the string. This means that every output will be start on a new line. Always remember that every java statement must end with a semicolon.

JAVA TOKENS

The smallest individual units in a program are known as tokens. A Java program is basically a collection of classes. There are five types of tokens in Java language. They are: Keywords, Identifiers, Literals, Operators and Separators.

- **Keywords:** Keywords are some reserved words which have some definite meaning. Java language has reserved 60 words as keywords. They cannot be used as variable name and they are written in lower-case letter. Since Java is case-sensitive, one can use these words as identifiers by changing one or more letters to upper-case. But generally it should be avoided. Java does not use many keywords of C/C++ language but it has some new keywords which are not present in C/C++. A list of Java keywords are given in the following table:

abstract	boolean	break	byte	byvalue
case	cast	catch	char	class
const	continue	default	do	double
else	extends	false	final	finally
float	for	future	generic	goto
if	implements	import	inner	instanceof
int	interface	long	native	new
null	operator	outer	package	private
protected	public	rest	return	short
static	super	switch	synchronized	this
threadsasafe	throw	throws	transient	true
try	var	void	volatile	while

• **Identifiers** : Java Identifiers are used for naming classes, methods, variables, objects, labels in a program. These are actually tokens designed by programmers. There are a few rules for naming the identifiers. These are:

- Identifier name may consist of alphabets, digits, dollar (\$) character, underscore(_).
- Identifier name must not begin with a digit
- Upper case and lowercase letters are distinct.
- Blank space is not allowed in an identifier name.
- They can be of any length. While writing Java programs, the following naming conventions should be followed by programmers :
 - * All local and private variables use only lower-case letters. Underscore is combined if required. For example, total_marks average
 - * When more than one word is used in a name, the second and subsequent words are marked with a leading upper-case letter. For example, dateOfBirth, totalMarks, studentName
 - * Names of all public methods and interface variables start with a leading lower-case letter. For example, total, average
 - * All classes and interfaces start with a leading upper-case letter. For example, HelloJava Employee ComplexNumber
 - * Variables that represent constant values use all upper-case letters and underscore between words if required. For example, PI RATE MAX_VALUE
- **Literals**: Literals in Java are a sequence of characters such as digits, letters and other characters that represent constant values to be stored in a variable.

- **Operators:** An operator is a symbol that takes one or more arguments and operates on them to produce a result.
- **Separators:** Separators are symbols used to indicate where groups of code are arranged and divided. Java separators are as follows: { } Braces () Parentheses [] Brackets ; Semicolon , Comma . Period

VARIABLES

A variable is an identifier that denotes a storage location where a value of data can be stored. The value of a variable in a particular program may change during the execution of the program.

THE SCOPE OF VARIABLES

In addition to the name and the type that we explicitly give to a variable, a variable has scope. The section of code where the variable's simple name can be used is the variable's scope. The variable's scope is determined implicitly by the location of the variable declaration, that is, where the declaration appears in relation to other code elements. We will learn more about the scope of Java variables in this section. Java variables are categorized into three groups according to their scope. These are: local variable, instance variable and class variable.

- Local variables are variables which are declared and used inside methods. Outside the method definition they are not available for use and so they are called local variables. Local variables can also be declared inside program blocks that are defined between an opening { and a closing brace }. These variables are visible to the program only.
- Instance variables are created when the class objects are instantiated. They can take different values for each object.
- Class variables are global to a class and belong to the entire set of object that class creates. Only one memory location is reserved for each class variable. Instance and class variables are declared inside a class.

CONSTANTS

While a program is running, different values may be assigned to a variable at different times (thus the name variable, since the values it contains can vary), but in some cases we don't want this to happen. If we want a value to remain fixed, then we use a constant. Constants in Java refer to fixed values that don't change during the execution of a program. A constant is declared in a manner similar to a variable but with additional

reserved word final. A constant must be assigned a value at the time of its declaration. Thus, the general syntax is:

final type name = value ;

An example of declaring some constants:

final double PI = 3.14159;

final int PASS_MARK = 200 ;

constants

Java supports several types of constants such as Integer constants, Real constants, Single Character constants, String constants, Backslash Character constants.

- **Integer constant** : An integer constant refers to a sequence of digits. For example, Decimal integer constants: 5, -5, 0, 254321 Octal: 0, 025, 0125 Hexadecimal: 0X2, 0X9F, 0x etc.

- **Real constant** : Real or floating point constants are represented by numbers containing fractional parts. For example, 0.125, -.25, 124.75, .8, 450., -85 etc. Exponential notation: 1.5e+4, 0.55e4 etc.

- **Single Character constant** : Character constant contains a single character enclosed within a pair of single quote marks. For example, '1', '15', 'A', 'c', ' ', ' ; ' etc.

- **String constant** : It is a sequence of characters enclosed between double quotes. Characters may be alphabets, digits, special characters, blank spaces. For example, "2010", "KKSHOU", "3+4", "Hello Java", "\$5", "A"

- **Backslash Character constant** : Java supports some backslash character constants that are used in output methods. These are also known as escape sequences. For example,

'\b' back space

'\n' new line

'\f' form feed

'\r' carriage return

'\t' horizontal tab

'\'' single quote

'\"' double quote

'\\' backslash

Unicode System

Unicode is a universal international standard character encoding that is capable of representing most of the world's written languages.

Before Unicode, there were many language standards:

ASCII (American Standard Code for Information Interchange) for the United States.

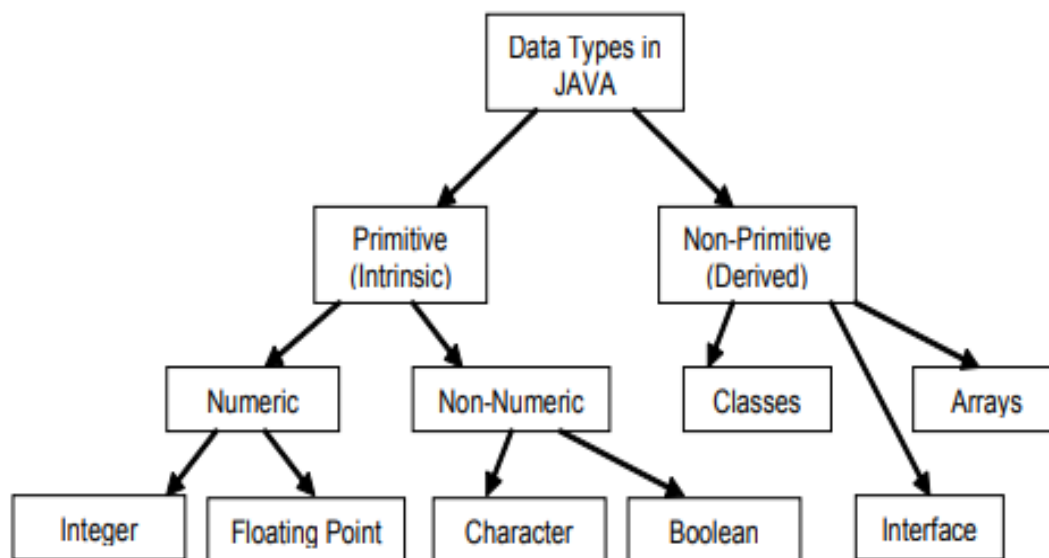
ISO 8859-1 for Western European Language.

KOI-8 for Russian.

GB18030 and BIG-5 for Chinese, and so on.

DATA TYPES

Every variable must have a data type. A data type determines the values that the variable can contain and the operations that can be performed on it. A variable's type also determines how its value is stored in the computer's memory.



A variable of primitive type contains a single value of the appropriate size and format for its type: a number, a character, or a boolean value.

Primitive types are also termed as intrinsic or built-in types. The primitive types are described below:

- Integer type Integer type can hold whole numbers like 1,2, 3,... -4, 1996 etc. Java supports four types of integer types: byte, short, int and long. It does not support unsigned types and therefore all Java values are signed types. This means that they can be positive or negative. For example, the int value 1996 is actually stored as the bit pattern 00000000000000000000000011111001100 as the binary equivalent of 1996 is 11111001100. Similarly, we must use a byte type variable for storing a number like 20 instead of an int type. It is because that smaller data types require less time for

manipulation. We can specify a long integer by putting an 'L' or 'l' after the number. 'L' is preferred, as it cannot be confused with the digit '1'.

- **Floating Point type** Floating point type can hold numbers containing fractional parts such as 2.5, 5.75, -2.358. i.e., a series of digits with a decimal point is of type floating point. There are two kinds of floating point storage in Java. They are: float (Single-precision floating point) and double (Double-precision floating point). In general, floating point numbers are treated as double-precision quantities. To force them to be in single-precision mode, we must append 'f' or 'F' to the numbers. For example, 5.23F, 2.25f
- **Character type** Java provides a character data type to store character constants in memory. It is denoted by the keyword char. The size of char type is 2 bytes.
- **Boolean type** Boolean type can take only two values: true or false. It is used when we want to test a particular condition during the execution of the program. It is denoted by the keyword boolean and it uses 1 byte of storage.

The memory size and range of all eight primitive data types are given

Type	Size	Minimum Value	Maximum Value
byte	1 byte	-128	127
short	2 bytes	-32,768	32,767
int	4 bytes	-2,147,483,648	2,147,483,647
long	8 bytes	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
float	4 bytes	3.4e-038	3.4e+038
double	8 bytes	1.7e-308	1.7e+308
char	2 bytes	a single Unicode character	
boolean	1 byte	a boolean value (true or false)	

In addition to eight primitive types, there are also three kinds of non-primitive types in JAVA. They are also termed as reference or derived types. The non-primitive types are: arrays, classes and interfaces. The value of a non-primitive type variable, in contrast to that of a primitive type, is a reference to (an address of) the value or set of values represented by the variable. These are discussed later as and when they are encountered.

OPERATORS AND EXPRESSIONS

Operators are special symbols that are commonly used in expressions. An operator performs a function on one, two, or three operands. An operator that requires one operand is called a unary operator. For example, ++ is a unary operator that increments the value of its operand by 1. An operator that requires two operands is a binary operator. For example, = is a binary operator that assigns the value from its righthand

operand to its left-hand operand. And finally, a ternary operator is one that requires three operands. The Java programming language has one ternary operator (?:) .

Expressions are the simplest form of statement in Java that actually accomplishes something. Expressions are statements that return a value. Many Java operators are similar to those in other programming languages. Java supports most C++ operators. In addition, it supports a few that are unique to it.

Operators in Java include

1. arithmetic,
2. assignment,
3. increment and decrement,
4. Relational
5. logical operations.

Arithmetic Operators : Java has five operators for basic arithmetic

Operator	Meaning	Example
+	Addition	2 + 3
-	Subtraction	5 - 2
*	Multiplication	2 * 3
/	Division	14 / 4
%	Modulus	14 % 4

```
//Program 1: OperatorDemo.java
public class OperatorDemo
{
    public static void main(String[ ] args){
        int a = 5, b =3;
        double x = 25.50, y = 5.25;
        System.out.println("Variable values are :\n");
        System.out.println(" a = " + a);
        System.out.println(" b = " + b);
        System.out.println(" x = " + x);
        System.out.println(" y = " + y);
        //Addition
        System.out.println("\nAddition...");
        System.out.println(" a + b = " + (a + b));
```

```
System.out.println(" x + y = " + (x + y));
//Subtraction
System.out.println("\nSubtraction...");
System.out.println(" a - b = " + (a - b));
System.out.println(" x - y = " + (x - y));
//Multiplication
System.out.println("\nMultiplication...");
System.out.println(" a * b = " + (a * b));
System.out.println(" x * y = " + (x * y));
//Division operation
System.out.println("\nDivision...");
System.out.println(" a / b = " + (a / b));
System.out.println(" x / y = " + (x / y));
//Modulus operation
System.out.println("\nModulus...");
System.out.println(" a % b = " + (a % b));
System.out.println(" x % y = " + (x % y));
}
}
```

Assignment Operators :

Assignment operators are used to assign the value of an expression to a variable. The usual assignment operator is '='. The general syntax is:

variableName = value;

For example, sum = 0; // 0 is assigned to the variable sum

x = x + 1;

Like C/C++, Java also supports the shorthand form of assignments. For example, the statement x = x + 1; can be written as x += 1; in shorthand form.

Increment and Decrement Operators :

The unary increment and decrement operators ++ and -- comes in two forms, prefix and postfix. They perform two operations. They increment (or decrement) their operand, and return a value for use in some larger expression.

In prefix form, they modify their operand and then produce the new value. In postfix form, they produce their operand's original value, but modify the operand in the background. For example, let us take the following two expressions:

y = x++;

y = ++x;

These two expressions give very different results because of the difference between prefix and postfix. When we use postfix operators (`x++` or `x--`), `y` gets the value of `x` before `x` is incremented; using prefix, the value of `x` is assigned to `y` after the increment has occurred.

Relational Operators :

Java has several expressions for testing equality and magnitude. All of these expressions return a boolean value (that is, true or false). the relational operators:

Operator	Meaning	Example
<code>==</code>	Equal	<code>x == 5</code>
<code>!=</code>	Not equal	<code>x != 10</code>
<code><</code>	Less than	<code>x < 7</code>
<code>></code>	Greater than	<code>x > 4</code>
<code><=</code>	Less than or equal to	<code>y <= 8</code>
<code>>=</code>	Greater than or equal to	<code>z >= 15</code>

Logical Operators :

Expressions that result in boolean values (for example, the Relational operators) can be combined by using logical operators that represent the logical combinations AND, OR, XOR, and logical NOT.

For AND operation, the `&&` symbol is used. The expression will be true only if both operands tests are also true; if either expression is false, the entire expression is false.

For OR expressions, the `||` symbol is used. OR expressions result in true if either or both of the operands is also true; if both operands are false, the expression is false.

In addition, there is the XOR operator `^`, which returns true only if its operands are different (one true and one false, or vice versa) and false otherwise (even if both are true).

For NOT, the `!` symbol with a single expression argument is used. The value of the NOT expression is the negation of the expression; if `x` is true, `!x` is false.

Bitwise Operators :

Bitwise operators are used to perform operations on individual bits in integers. Table 2.5 summarizes the bitwise operators available in the JAVA programming language. When both operands are boolean, the bitwise AND operator (`&`) performs the same operation as logical AND (`&&`). However, `&` always evaluates both of its operands and returns

true if both are true. Likewise, when the operands are boolean, the bitwise OR (`|`) performs the same operation as is similar to logical OR (`||`). The `|` operator always evaluates both of its operands and returns true if at least one of its operands is true. When their operands are numbers, `&` and `|` perform bitwise manipulations

Operator	Meaning
<code>&</code>	Bitwise AND
<code> </code>	Bitwise OR
<code>^</code>	Bitwise XOR
<code><<</code>	Left shift
<code>>></code>	Right shift
<code>>>></code>	Zero fill right shift
<code>~</code>	Bitwise complement
<code><<=</code>	Left shift assignment (<code>x = x << y</code>)
<code>>>=</code>	Right shift assignment (<code>x = x >> y</code>)
<code>>>>=</code>	Zero fill right shift assignment (<code>x = x >>> y</code>)
<code>x&=y</code>	AND assignment (<code>x = x & y</code>)
<code>x =y</code>	OR assignment (<code>x = x y</code>)
<code>x^=y</code>	NOT assignment (<code>x = x ^ y</code>)

A shift operator performs bit manipulation on data by shifting the bits of its first operand right or left. For example if `op1` and `op2` are two operands, then the statement

`op1 << op2;`

shift bits of `op1` left by distance `op2`; fills with zero bits on the righthand side and `op1 >> op2;` shift bits of `op1` right by distance `op2`; fills with highest (sign) bit on the left-hand side.

`op1 >>> op2;`

shift bits of `op1` right by distance `op2`; fills with zero bits on the lefthand side. Each operator shifts the bits of the left-hand operand over by the number of positions indicated by the right-hand operand. The shift occurs in the direction indicated by the operator itself. For example, the statement `25 >> 1;` shifts the bits of the integer 25 to the right by one position. The binary representation of the number 25 is 11001. The result of the shift operation of 11001 shifted to the right by one position is 1100, or 12 in decimal

The Java programming language also supports the following Operators

Operator	Description
?:	Conditional operator (a ternary operator)
[]	Used to declare arrays, to create arrays, and to access array elements
.	Used to form qualified names
(params)	Delimits a comma-separated list of parameters
(type)	Casts (converts) a value to the specified type
new	Creates a new object or array
instanceof	Determines whether its first operand is an instance of its second operand

CONTROL FLOW STATEMENTS

When we write a program, we type statements into a file. Without control flow statements, the compiler executes these statements in the order they appear in the file from left to right, top to bottom in a sequence. We can use control flow statements in our programs to conditionally execute statements, to repeatedly execute a block of statements, and to otherwise change the normal, sequential flow of control.

Flow control in Java uses similar syntax as in C and C++. The Java programming language provides several control flow statements, which are listed below :

- Decision making : if, if-else, switch-case
- Looping : while, do-while, for
- Branching : break, continue, label :, return

Decision Making Statements While programing, we have a number of situations where we may have to change the order of execution of statements based on certain cosditions/decisions. This involves a kind of decision making to see whether a particular condition has occured or not and then direct the computer to execute certain statements accordingly. The statements used to handle those situation are called decision making statement. The if and if-else Statements The if statement enables our program to selectively execute other statements, based on some criteria.

The syntax of if statement is:

```
if (boolean_expression)
{
```

```
statement-block ;  
}
```

```
statement;
```

The statement-block may be a single statement or a group of statements. If the boolean-expression evaluates to true, then the block of code inside the if statement will be executed. If not the first set of code after the end of the if statement(after the closing curly brace) will be executed. For example,

```
if (percentage>=40)  
{  
    System.out.println("Pass");  
}
```

In this case, if percentage contains a value that is greater than or equal to 40, the expression is true, and println() will execute. If percentage contains a value less than 40, then the println() method is bypassed. What if we want to perform a different set of statements if the expression is false? We use the else statement for that.

The general syntax of if-else statement is:

```
if ( Boolean_expression )  
    statement; //executes when the expression is true  
else  
    statement; //executes when the expression is false
```

Let us consider the same example but at this time the output should be Pass or Fail depending on percentage of marks. i.e., if percentage is equal to or more than 40 then the output should be Pass; otherwise Fail. This can be done by using an if statement along with an else statement. Here is the segment of code :

```
if (percentage>=40)  
    System.out.println("Pass");  
else  
    System.out.println("Fail");
```

When a series of decisions are involved, we may have to use more than one if-else statements in nested form. The switch statement We have seen that when one of the many alternatives is to be selected, we can design a program using if statements to control the selection. However, the program becomes difficult to read and follow when the number of alternatives increases. Like C/C++, JAVA has a built-in multiway decision statement known as a switch. The switch statement provides variable entry points to a block. It tests the value of a given variable or expression against a list of case

values and when a match is found, a block of statements associated with that case is executed. The general form of switch statement is as follows :

```
switch( expression )
{
case value : statements;
    break;
case value : statements
    break;
...
default : statements // optional default section
    break;
}
```

The expression is evaluated and compared in turn with each value prefaced by the case keyword. The values must be constants (i.e., determinable at compile-time) and may be of type byte, char, short, int, or long.

Looping

In looping, a sequence of statements are executed until some conditions for the termination of the loop are satisfied. The process of repeatedly executing a block of statements is known as looping. At this point, we should remember that Java does not support goto statement. Like C/C++, Java also provides the three different statements for looping. These are:

- while
- do-while
- for

The while and do-while statements We use a while statement to continually execute a block while a condition remains true. The general syntax of the while statement is:

```
while(expression)
{
    statement(s);
}
```

First, the while statement evaluates expression , which must return a boolean value. If the expression returns true, the while statement executes the statement(s) in the while block. The while statement continues testing the expression and executing its block until the expression returns false. The Java programming language provides another statement

that is similar to the while statement : the do-while statement. The general syntax of do-while is:

```
do
{
    statement(s);
}while (expression);
```

Statements within the block associated with a do-while are executed at least once. Instead of evaluating the expression at the top of the loop, do-while evaluates the expression at the bottom. Here is the previous program rewritten to use do-while loop

A program of while loop is shown below :

```
class Fibo
{
    public static void main(String args[])
    {
        System.out.println("0\n1");
        int n0=0,n1=1,n2=1;
        while(n2<50)
        {
            System.out.println(n2);
            n0=n1;
            n1=n2;
            n2=n1+n0;
        }
        System.out.println(n2);
    }
}
```

The for statement The for statement provides a compact way to iterate over a range of values. The general form of the for statement can be expressed like this:

```
for (initialization; termination_condition; increment)
{
    statement(s);
}
```

The initialization is an expression that initializes the loop. It is executed once at the beginning of the loop. The termination_condition determines when to terminate the loop. This condition is evaluated at the top of each iteration of the loop. When the condition

evaluates to false, the loop terminates. Finally, increment is an expression that gets invoked after each iteration through the loop. All these components are optional. In fact, to write an infinite loop, we can omit all three expressions: `for ( ; ; )`

```
{  
    // infinite loop  
}
```

Often, for loops are used to iterate over the elements in an array or the characters in a string. The following program segment uses a for loop to calculate the summation of 1 to 50: `for (i =1; i <= 50 ; i + +)`

```
{  
    sum = sum + i ;  
}
```

BRANCHING STATEMENTS

The Java programming language supports three branching statements:

- The break statement
- The continue statement
- The return statement

Break

The break statement In JAVA, the break statements has two forms: unlabelled and labelled. We have seen the unlabelled form of the break statement used with switch earlier. As noted there, an unlabelled break terminates the enclosing switch statement, and the flow of control transfers to the statement immediately following the switch. It can be used to terminate a for, while, or do-while loop. A break (unlabelled form) statement, causes an immediate jump out of a loop to the first statement after its end. When the break statement is encountered inside a loop, the loop is immediately exited and the program continues with the statement immediately following the loop. When the loop is nested, the break would only exit from the loop containing it. This means, the break will exit only a single loop.

The continue statement

The continue statement causes an immediate branch to the end of the innermost loop that encloses it, skipping over any intervening statements. It is written as:

```
continue ;
```

A continue does not cause an exit from the loop. Instead, it immediately initiates the next iteration. We can use the continue statement to skip the current iteration of a for, while, or do-while loop. In Java, we can give a label to a block of statements. A label is any valid Java variable name. To give a label to a loop, we have to place the label name before the loop with a colon at the end. For example,

```
loop1: for( ..... )
{
    .....
    .....
}
```

..... We have seen that a simple break statement causes the control to jump outside the nearest loop and a simple continue statement returns the current loop. If we want to jump outside a nested loop or to continue a loop that is outside the current one, then we may have to use the labelled break and labelled continue statement. The labelled form of break and continue can be used as follows:

```
breakDemo.java
class breakDemo
{
    public static void main(String[] args)
    {
        outer: for(int i=1;i<100;i++)
        {
            System.out.println(" ");
            if(i>=10)
                break;
            for( int j=1;j<100;j++)
            {
                System.out.print("* ");
                if(j==i)
                    continue outer; //labelled continue
            }
        }
    }
}
```

The return statement

The last of the branching statements is the return statement. We can use return to exit from the current method. The flow of control returns to the statement that follows the original method call. The return statement has two forms: one that returns a value and one that does not.

To return a value, simply put the value (or an expression that calculates the value) after the return keyword: `return sum;` The data type of the value returned by return must match the type of the method's declared return value. When a method is declared void, use the form of return that does not return a value:

`return;`

Anything we wish to represent in a Java program must be encapsulated in a class. Classes are the building blocks of a Java application. In this unit, we will explore the object-oriented aspects of Java. We will learn about the concept of classes in Java, how to access class members, how to create instance of classes etc. Some other important concepts like inheritance, super classes, constructors etc. are also covered in this unit. At the end of this unit, we will acquaint you with the concept of recursion.

Java Comments

The Java comments are the statements in a program that are not executed by the compiler and interpreter.

Comments are used to make the program more readable by adding the details of the code.

It makes easy to maintain the code and to find the errors easily.

The comments can be used to provide information or explanation about the variable, method, class, or any statement.

It can also be used to prevent the execution of program code while testing the alternative code.

Types of Java Comments

There are three types of comments in Java.

- Single Line Comment
- Multi Line Comment
- Documentation Comment

CLASS FUNDAMENTALS

Classes provide a convenient method for grouping together logically related data items and functions that work on them. A class can contain methods or functions, variables, initialization code etc. In case of Java, the data items are termed as fields and the functions are termed as methods.

Class serves as a blueprint for making class instances, which are runtime objects that implement the class structure. Thus, an object is defined to be an instance of a class. An object consists of attributes and behaviors. An attribute is a feature of the object, something the object “has.” A behavior is something the object “does”.

DECLARING AND DEFINING A CLASS

A class in Java is declared using the class keyword. A source code file in Java can contain exactly one public class, and the name of the file must match the name of the public class with a . java extension. We can declare more than one class in a . java file, but at most one of the classes can be declared public. The name of the source code file must still match the name of the public class. If there are no public classes in the source code file, the name of the file is arbitrary. The general form of a class definition is as follows:

```
class classname
{
    type instanceVariable1;
    type instanceVariable2;
    .....
    .....
    type instanceVariableN;
    type methodName1(parameterList)
    {
        // body of method
    }
    type methodName2(parameterList)
    {
        // body of method
    }
    .....
    .....
    type methodNameN(parameterList)
    {
        // body of method
    }
}
```

Here, we can see that, there is no semicolon after closing brace of class in Java. But in case of C++ language, class definition ends with a semicolon. The data or variables, defined within a class are called fields or instance variables as each instance of the class

(that is, each object of the class) contains its own copy of these variables. Thus, the data for one object is separate and unique from the data for another. The methods are used for manipulating the instance variables(data items) contained in the class. The methods and variables defined within a class are collectively called members of the class. A class may contain three kinds of members: fields, methods and constructors. Constructor specify how the objects are to be created. Java classes do not need to have a main() method. The general form of a class does not specify a main() method. We only specify one if that class is the starting point for our program. The fields and methods of a class appear within the curly brackets of the class declaration. A class may not contain anything inside the curly brace { and }. Thus we can have empty classes in Java. An empty class cannot do anything as it does not contain any properties. But it is possible to create objects of such classes. The structure of an empty class is as follows:

```
class classname
{
}
```

Let us consider another class Triangle which has two float type instance variables base and height.

```
class Triangle
{
    float base; //instance variable base
    float height; //instance variable height
}
```

Until object of Triangle class is created, there is no storage space has been created in the memory for these two variables base and height. When an object of Triangle class is created, memory will be allocated for each of these two fields.

ADDING METHODS TO A CLASS

A method is a sequence of declarations and executable statements encapsulated together like independent mini program. For the manipulation of data fields inside a class we have to add methods into the class. Method contains local variable declarations and other Java statements that are executed when the method is invoked.

- The name of the method, which can be any valid identifier
- Return value (i.e., the type of the value the method returns)
- List of parameters, which appears within parentheses
- Definition of the method (i.e., the body of the method)

A Java application must contain a main() method whose signature looks like this :

```
public static void main(String args[ ])
{
    // body;
}
```

The meaning of the keywords public, static etc. are already outlined in Unit-1. void indicates that the main() method has no return value. In Java, the definition (often referred to as the body) of a method must appear within the curly brackets that follow the method declaration. For example, let us add two methods in the Triangle class.

```
class Triangle
{
    float base, height;
    void readData(float b, float h) //definition of method
    {
        base=b;
        height=h;
    }
    float findArea( )//definition of another method
    {
        float area = 0.5*(base *height);
        return area;
    }
}
```

In the above code, the Triangle class demonstrates how to add methods to a class. In the above class there are two methods, readData() and findArea(). The method readData() has a return type of void because it does not return any value. We pass two float type values to the method which are then assigned to the instance variables base and height. The method findArea() calculates area of the triangle and returns the result. Here, we can directly use base and height inside the method readData() and findArea(). Let us consider another method primeCheck() for the illustration of method. The following program tests a boolean method that checks its arguments for primality. The main() method prints those integers for which the checkPrime() returns true:

```
checkPrime.java
class checkPrime
{
    public static void main(String [ ] args)
    {
```

```
for(int i=0;i<50;i++)
if(isPrime(i))
System.out.print(i+ " ");
System.out.println();
}
static boolean isPrime(int n) //static method
{
if(n<2)
return false;
if(n==2)
return true;
if(n%2==0)
return false;
for(int j=3;j<=Math.sqrt(n);j=j+2)
if(n%j==0)
return false;
return true;
}
}
```

CREATING OBJECTS

It is important to remember that a class declaration only creates a template; it does not create an actual object. Java allocates storage for an object when we create it with the new operator. Creating an object is also referred to as instantiating an object. The new operator creates an object of the specified class and returns a reference to that object.

Thus, the preceding code does not cause any objects of type Triangle to come into existence. To create a Triangle object, we will use a statement like the following:

```
Triangle t1 = new Triangle( ); //t1 is an object of Triangle class
```

Here, the Triangle() is the default constructor of the class. The above statement can also be written by splitting it into two statements. This can be written as follows :

```
Triangle t1; // declares variable t1 to hold the object reference
t1 = new Triangle(); //assigns the object reference to the
//variable t1
```

Again, each time we create an instance of a class, we are creating an object that contains its own copy of each instance variable defined by the class. Thus, every Triangle object will contain its own copies of the instance variables base and height.

ACCESSING CLASS MEMBERS

Now we can say that when an object is created, each object contains its own set of variables. After creating objects of a class, we should assign values to these variables in order to use them in our program. Instance variables and methods cannot be accessible directly outside the class. For this, we have to use concerned object and the dot(.) operator. The dot operator links the name of the object with the name of an instance variable or method. The syntax of accessing class member is as follows:

```
objectName.instanceVariableName ;  
objectName.methodName(parameterList);
```

For example, suppose we have created t1 object with the following statement

```
Triangle t1 = new Triangle( );
```

Now to assign the value 20.5 to the base variable of t1, we would use the following statement:

```
t1.base = 20.5;
```

This statement tells the compiler to assign the copy of base that is contained within the t1 object the value of 20.5. Here is a complete program that uses the Triangle class.

```
: Area.java  
class Triangle  
{  
    double base, height;  
    void readData(double b, double h) //definition of method  
    {  
        base=b;  
        height=h;  
    }  
    double findArea( )//definition of another method  
    {  
        double a = 0.5*(base *height);  
        return a;  
    }  
}  
class Area //class with the main() method  
{  
    public static void main(String args[ ])  
    {  
        double area1, area2;
```

```
Triangle t1=new Triangle();
Triangle t2=new Triangle();
t1.base=20.5;
t1.height=15.5;
area1= t1.base * t1.height;
t2.readData(30.0,15.0);
area2=t2.findArea();
System.out.println("Area of the first triangle is = "+area1);
System.out.println("\nArea of the second triangle is = "+area2);
}
}
```

The Triangle class has no main() method. So it cannot be executed as Java program. We need a separate class that does have a main() method. In the above program, the class Area contains the main() method. The name of the file where we write the program should have the name Area.java .

If we write these two classes Triangle and Area into two separate files namely Triangle.java and Area.java, then these two files should be saved in the same folder in the computer. To compile Area.java program, we must first compile the Triangle class as:

```
javac Triangle.java
```

This will create the bytecode file Triangle.class in the same folder. Now, we can compile and execute the Area.java file with the following statements:

```
javac Area.java
java Area
```

If we look at the contents of the folder, we will find four files: the source code and bytecode for Triangle and the source code and bytecode for Area. The statements

```
Triangle t1=new Triangle();
Triangle t2=new Triangle();
```

will create two Triangle objects t1 and t2 in memory, which reserves memory for two base fields , two height fields, two readData() methods and two findArea() methods. They are distinguished by which reference we use. Each of the fields is initialized in both objects using the dot operator with the following statements:

```
t1.base=20.5;
t1.height=15.5;
```

The Triangle reference t1 points to the Triangle object whose base is 20.5 and height is 15.5. The base and height field of t2 object is initialized to 30.0 and 15.0 with the statement t2.readData(30.0,15.0); To compute the area of the triangle t1, we can use any one of the following statements:

```
area1= t1.base * t1.height;
```

```
area1=t1.findArea();
```

Similarly, for t2, we can use

```
area2=t2.base * t2.height;
```

```
area2=t2.findArea();
```

CONSTRUCTORS

It would be simpler and more concise to initialize an object when it is first created. Java supports constructors that enable an object to initialize itself when it is created. This section provides a brief introduction about the Constructor and how constructors are overloaded in Java. Constructor is always called by new operator. Constructors are declared just like as we declare methods, except that the constructor does not have any return type. The name of the constructors must be the same with the class name where it is declared. It is called when a new class instance is created, which gives the class an opportunity to set up the object for use. Constructors, like other methods, can accept arguments and can be overloaded but they cannot be inherited. For example, let us consider the same Triangle class. We can replace the readData() method by a constructor. This can be accomplished as follows:

TriangleArea.java

```
class Triangle
```

```
{
```

```
    double base, height;
```

```
    Triangle(double b, double h) //constructor with two arguments
```

```
    {
```

```
        base=b;
```

```
        height=h;
```

```
    }
```

```
    double findArea( ) //definition of method findArea()
```

```
    {
```

```
        double area = 0.5*(base *height);
```

```
        return area;
```

```
    }
```

```
}
```

```
class TriangleArea //class with the main() method
```

```
{
```



```
public static void main(String args[ ])
{
    Triangle t1=new Triangle(20.5, 15.5); // call of constructor
    double area1= t1.findArea();
    System.out.println("Area of the triangle is = "+area1);
}
}
```

The output of the program will be:

Area of the triangle is 317.75

The Triangle class contains a single constructor. We can recognize a constructor because its declaration uses the same name as the class and it has no return type. The constructor in the Triangle class takes two arguments. The statement Triangle t1= new Triangle(20.5, 15.5); provides 20.5 and 15.5 as values for those arguments.

DEFAULT AND COPY CONSTRUCTOR

There are two special kinds of constructors that a class may have: a default constructor and a copy constructor.

The default constructor is what gets called whenever we create an object by calling its constructor with no arguments. If we don't define a constructor for a class, a default constructor is automatically created by the compiler. It initializes all instance variables to default value (zero for numeric types, null for object references, and false for booleans). There may be only one default constructor in a class. The copy constructor is a constructor whose only parameter is a reference to an object of the same class to which the constructor belongs. It is called copy constructor as it is used to duplicate an existing object of the class.

CircleArea.java

```
class Circle
{
    double radius;
    Circle() //default constructor
    {
        radius =10.0;
    }
    Circle(Circle c) //copy constructor
    {
        radius = c.radius;
    }
    double findArea( ) //for calculation of area
```

```
{
double area = 3.14 *radius *radius;
return area;
}
}
class CircleArea //class with the main() method
{
public static void main(String args[ ])
{
double p_area,q_area;
Circle p = new Circle(); //invokes the default constructor
Circle q = new Circle(p); //invokes copy constructor
p_area= p.findArea();
q_area =q.findArea();
System.out.println("Area of circle p = "+p_area);
System.out.println("\nArea of circle q = "+q_area);
}
}
```

The statement `Circle p = new Circle();` invokes the default constructor and it creates a `Circle` object `p` with radius 10.0. The statement `Circle q = new Circle(p);` invokes the copy constructor and it also creates another `Circle` object `q` with the same radius value 10.0. The object `p` and `q` are two separate but equal objects. In the output we can see that the area of both the circles are same. All fields in a class will automatically be initialized with their type's default values unless the constructor explicitly uses other values.

OVERLOADING OF CONSTRUCTORS

Overloading of constructors means multiple constructors in a single class. Program 3.4 is also an example of constructor overloading as there are two constructors in that program. Constructor can be overloaded provided they should have different arguments because Java compiler differentiates constructors on the basis of arguments passed in the constructor. Let us consider the following `Rectangle` class for the demonstration of constructor overloading. After you going through it the concept of constructor overloading will be more clear. to you. In the example below we have written four constructors each having different arguments types.

ConstructorOverloading.java

```
class Rectangle
{
```

```
int l, b; // for int type length and breadth
float p, q; // for float type length and breadth
Rectangle(int x, int y) // two int type argument constructor
{
    l = x;
    b = y;
}
int first() { //method
    return(l * b);
}
Rectangle(int x) { //one int type argument constructor
    l = x;
    b = x;
}
int second() { //method
    return(l * b);
}
Rectangle(float x) { //one float type argument constructor
    p = x;
    q = x;
}
float third()
{
    return(p * q);
}
Rectangle(float x, float y) {
    p = x;
    q = y;
}
float fourth() {
    return(p * q);
}
}
class ConstructorOverloading {
    public static void main(String args[ ]) {
        Rectangle r1=new Rectangle(2,4);
        int area1=r1.first();
        System.out.println(" Area in first constructor : “ + area1);
        Rectangle r2=new Rectangle(5);
```

```
int area2=r2.second();
System.out.println("\nArea in second constructor: “ + area2);
Rectangle r3=new Rectangle(2.0f);
float area3=r3.third();
System.out.println("\nArea in third constructor: “ + area3);
Rectangle r4=new Rectangle(3.0f,2.0f);
float area4=r4.fourth();
System.out.println("\nArea in fourth constructor: “ + area4);
}
}
```

Constructor can also invoke other constructors with the `this` and `super` keywords. We will discuss the first case here, and return to that of the superclass constructor after we have talked more about subclassing and inheritance. A constructor can invoke another, overloaded constructor in its class using the reference `this()` with appropriate arguments to select the desired constructor. If a constructor calls another constructor, it must do so as its first statement: `this()` calls another constructor in same class. Often a constructor with few parameters will call a constructor with more parameters, giving default values for the missing parameters. We can use `this` to call other constructors in the same class.

```
class Triangle
{
    double base, height;
    Triangle(double b,double h) {
        base = b;
        height=h;
    }
    Triangle(double b)
    {
        this(b, 20.50 );
    }
}
```

In the above code, the class `Triangle` has two constructors. The first, accepts arguments specifying the triangles's base and height. The second constructor takes just the base as an argument and, in turn, calls the first constructor with a default value 20.50 for height. We have considered a simple example for clear understanding but the advantage of this approach is that we can have a single constructor do all the complicated setup work; other auxiliary constructors simply feed the appropriate arguments to that constructor.

The call to this() must appear as the first statement in our second constructor. It should be remembered that we can invoke a second constructor only as the first statement of another constructor.

PASSING ARGUMENTS TO METHODS

In Java, we can pass an argument of any valid Java data type into a method similar to functions in other programming languages like C, C++ etc. This includes simple data types such as characters, integers, floats, doubles and boolean as well as complex types such as arrays, objects etc. Arguments provide information to the method from outside the scope of the method. A method in Java always specifies a return type. The returned value can be a primitive type, a reference type, or the type void , which indicates no returned value. The declaration for a method or a constructor declares the number and the type of the arguments for that method or constructor. For example, let us consider the following method that computes the area of a rectangle:

```
int computeArea(int width, int height) //method header
{
    int area; // area is a local variable
    area = width * height;
    return area;
}
```

The first int indicates that the value this method returns is going to be an integer. The name of the method is “computeArea”, and it has two integer parameters: length and breadth. The body of the method starts with the left brace, “{” and end with the right brace, “}”. The method is returning the area with the return statement. For calling the method, we can write :

```
int a= r1.computeArea(16,8); //method call
```

- **Formal and Actual parameter :** We use the term formal parameters to refer to the parameters in the definition of the method. In the example, width and height are the formal parameters. We use the term actual parameters to refer to variables in the method call. They are called “actual” because they determine the actual values that are sent to the method. Let us consider what happens when we pass arguments to a method. In Java, all primitive data types (e.g., int, char, float) are passed by value. The reference types (i.e., any kind of object, including arrays and strings) are used through references. An important distinction is that the references themselves (the pointers to these objects) are actually primitive types and are passed by value too.

- **Pass-by-Value** Pass-by-value means that when we call a method, a copy of the value of each actual parameter is passed to the method. We can change that copy inside the method, but this will have no effect on the actual parameter. Unlike many other languages, Java has no mechanism for changing the value of an actual parameter. Java has eight primitive data types. Let us consider the following example for the demonstration of pass-by-value:

PassPrimitiveByValue.java

```
class PassPrimitiveByValue
{
    public static void main(String[ ] args)
    {
        int p = 3;
        System.out.println("Before calling passValue, p = " + p);
        passValue(p); //call passValue() with p as argument
        System.out.println("\nAfter calling passValue, p = " + p);
    }
    public static void passValue(int p) //passValue() definition to
    { //change the value of parameter
        p = 10;
    }
}
```

Here, we can see that the outputs are same before and after calling the method `passValue()`. When Java calls a method, it makes a copy of its actual parameters' values and sends the copies to the method where they become the values of the formal parameters. Then when the method returns, those copies are discarded and the actual parameters have remained unchanged. Passing variables by value affords the programmer some safety. Methods cannot unintentionally modify a variable that is outside of its scope. However, we often want a method to be able to modify one or more of its arguments. In the `passValue()` method, the caller wants the method to change the value through its arguments. However, the method cannot modify its arguments, and, furthermore, a method can only return one value through its return value. So, it is necessary to learn how a method can return more than one value, or have an effect (modify some value) outside of its scope. To allow a method to modify a argument, we must pass in an object. Objects in Java are also passed by value; however, the value of an object is a reference. So, the effect is that the object is passed in by reference. When

passing an argument by reference, the method gets a reference to the object. A reference to an object is the address of the object in memory. Now, the local variable within the method is referring to the same memory location as the variable within the caller. For example, if a parameter to a method is an object reference. We can manipulate the object in any way, but we cannot make the reference refer to a different object.

: Record.java (demonstration of passing object to method)

```
class Record
{
    int roll;
    String name;
    static void tryObject(Record r) //parameter is an object
    { //reference
        r.roll = 1;
        r.name = "Anuj";
    }
    public static void main(String [] args)
    {
        Record obj = new Record(); //object obj of Record class
        obj.roll = 2;
        obj.name = "Rahul";
        System.out.println("Before calling tryObject(), the record is: "+
            obj.name + " " + obj.roll);
        tryObject(obj); //method call
        System.out.println("After calling tryObject(), the record is: " +
            obj.name + " " + obj.roll);
    }
}
```

The first print statement displays “Rahul 2”. The second print statement displays “Anuj 1”. Thus the object has been changed in this case. The reference to obj is the parameter to the method, so the method cannot be used to change that reference; i.e., it cannot make obj reference a different Record. But the method can use the reference to perform any allowed operation on the Record that it already references. It is often not good programming style to change the values of instance variables outside an object. Normally, the object would have a method to set the values of its instance variables

RECURSIVE METHOD Java supports recursion. Recursion is the process of defining something in terms of itself. As it relates to Java programming, recursion is the attribute

that allows a method to call itself. A method that calls itself is said to be recursive method. One of the suitable examples of recursion is the computation of the factorial of a number. The factorial of a number N is the product of all the whole numbers between 1 and N. for example, 4 factorial is $1 \times 2 \times 3 \times 4$, or 24. Here is how a factorial can be computed by use of a recursive method.

findFactorial.java

```
class findFactorial
{
    public static void main(String [] args)
    {
        for(int i=1;i<=10;i++)
            System.out.println("Factorial("+ i +") =" + fact(i)); //method call
    }
    static long fact(int n)//method definition
    {
        if(n<2)
            return 1;
        return n*fact(n-1); //Recursive call
    }
}
```

When fact() is called with an argument of 1, the function returns 1; otherwise it returns the product of $n * \text{fact}(n-1)$. To evaluate this expression, fact() is called with $n-1$. The main advantage of recursive methods is that they can be used to create clearer and simpler versions of several algorithms than can their iterative relatives.

Java static keyword

The **static keyword** in Java is used for memory management mainly. We can apply static keyword with variables, methods, blocks and nested classes. The static keyword belongs to the class than an instance of the class.

The static can be:

1. Variable (also known as a class variable)
2. Method (also known as a class method)
3. Block
4. Nested class

1) Java static variable

If you declare any variable as static, it is known as a static variable.

- The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc.
- The static variable gets memory only once in the class area at the time of class loading.

Advantages of static variable

It makes program memory efficient (i.e., it saves memory).

//Java Program to demonstrate the use of static variable

```
class Student{
    int rollno;//instance variable
    String name;
    static String college ="ITS";//static variable
    //constructor
    Student(int r, String n){
        rollno = r;
        name = n;
    }
    //method to display the values
    void display () {System.out.println(rollno+" "+name+" "+college);}
}
//Test class to show the values of objects
public class TestStaticVariable1 {
    public static void main(String args[]){
        Student s1 = new Student(111,"Karan");
        Student s2 = new Student(222,"Aryan");
        //we can change the college of all objects by the single line of code
        //Student.college="BBDIT";
        s1.display();
        s2.display();
    }
}
```

2) Java static method

If you apply static keyword with any method, it is known as static method.

- A static method belongs to the class rather than the object of a class.
- A static method can be invoked without the need for creating an instance of a class.

- A static method can access static data member and can change the value of it.

//Java Program to demonstrate the use of a static method.

```
class Student{
    int rollno;
    String name;
    static String college = "ITS";
    //static method to change the value of static variable
    static void change(){
        college = "BBDIT";
    }
    //constructor to initialize the variable
    Student(int r, String n){
        rollno = r;
        name = n;
    }
    //method to display values
    void display(){System.out.println(rollno+" "+name+" "+college);}
}

//Test class to create and display the values of object
public class TestStaticMethod{
    public static void main(String args[]){
        Student.change();//calling change method
        //creating objects
        Student s1 = new Student(111,"Karan");
        Student s2 = new Student(222,"Aryan");
        Student s3 = new Student(333,"Sonoo");
        //calling display method
        s1.display();
        s2.display();
        s3.display();
    }
}
```

this keyword in Java

Usage of Java this keyword

Here is given the 6 usage of java this keyword.

1. this can be used to refer current class instance variable.

2. this can be used to invoke current class method (implicitly)
3. this() can be used to invoke current class constructor.
4. this can be passed as an argument in the method call.
5. this can be passed as argument in the constructor call.
6. this can be used to return the current class instance from the method.

1) this: to refer current class instance variable

The this keyword can be used to refer current class instance variable. If there is ambiguity between the instance variables and parameters, this keyword resolves the problem of ambiguity.

```
class Student{
    int rollno;
    String name;
    float fee;
    Student(int rollno,String name,float fee){
        rollno=rollno;
        name=name;
        fee=fee;
    }
    void display(){System.out.println(rollno+" "+name+" "+fee);}
}
class TestThis1 {
    public static void main(String args[]){
        Student s1=new Student(111,"ankit",5000f);
        Student s2=new Student(112,"sumit",6000f);
        s1.display();
        s2.display();
    }}
}
```

2) this: to invoke current class method

You may invoke the method of the current class by using the this keyword. If you don't use the this keyword, compiler automatically adds this keyword while invoking the method. Let's see the example

```
class A{
    void m(){System.out.println("hello m");}
    void n(){
        System.out.println("hello n");
    }
}
```

```
//m();//same as this.m()
this.m();
}
}
class TestThis4{
public static void main(String args[]){
A a=new A();
a.n();
}}
```

3) this() : to invoke current class constructor

The this() constructor call can be used to invoke the current class constructor. It is used to reuse the constructor. In other words, it is used for constructor chaining.

```
class A{
A(){System.out.println("hello a");}
A(int x){
this();
System.out.println(x);
}
}
class TestThis5{
public static void main(String args[]){
A a=new A(10);
}}
```

Module II: Inheritance & Interface, Deriving Classes, Method Over-riding, Method Overloading, Access Modifiers, Abstract Class and Method, Interfaces, Packages, Imports and Class Path.

Inheritance in Java

Inheritance in Java is a mechanism in which one object acquires all the properties and behaviors of a parent object. It is an important part of OOPs (Object Oriented programming system).

Inheritance represents the IS-A relationship which is also known as a parent-child relationship.

Terms used in Inheritance

- Class: A class is a group of objects which have common properties. It is a template or blueprint from which objects are created.
- Sub Class/Child Class: Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- Super Class/Parent Class: Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.
- Reusability: As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in the previous class.

The syntax of Java Inheritance

```
class Subclass-name extends Superclass-name
```

```
{  
    //methods and fields  
}
```

The extends keyword indicates that you are making a new class that derives from an existing class. The meaning of "extends" is to increase the functionality.

The relationship between the two classes is Programmer IS-A Employee. It means that Programmer is a type of Employee.

```
class Employee{
    float salary=40000;
}
class Programmer extends Employee{
    int bonus=10000;
    public static void main(String args[]){
        Programmer p=new Programmer();
        System.out.println("Programmer salary is:"+p.salary);
        System.out.println("Bonus of Programmer is:"+p.bonus);
    }
}
```

Types of inheritance in java

On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.

In java programming, multiple and hybrid inheritance is supported through interface only.

Inheritance in Java is a concept that acquires the properties from one class to other classes; for example, the relationship between father and son.

In Java, a class can inherit attributes and methods from another class. The class that inherits the properties is known as the sub-class or the child class. The class from which the properties are inherited is known as the superclass or the parent class.

```
class derived_class extends base_class
{
    //methods
    //fields
}
```

Eg:1

```
class superclass
{
    // superclass data variables
    // superclass member functions
}
class subclass extends superclass
{
```

```
// subclass data variables
// subclass member functions
}
```

Inheritance uses the “extends” keyword to create a derived class by reusing the base class code.

Extends keyword in Java

The extended keyword extends a class and is an indicator that a class is being inherited by another class. When you say class B extends a class A, it means that class B is inheriting the properties(methods, attributes) from class A. Here, class A is the superclass or parent class and class B is the subclass or child class.

```
class Base
{
    public void M1()
    {
        System.out.println(“ Base Class Method ”);
    }
}
class Derived extends Base
{
    public void M2()
    {
        System.out.println(“ Derived Class Methods “);
    }
}
class Test
{
    public static void main(String[] args)
    {
        Derived d = new Derived(); // creating object
        d.M1(); // print Base Class Method
        d.M2(); // print Derived Class Method
    }
}
```

Terms Used in Inheritance

Class: A class is a collection of objects which have common properties.

Derived Class/Sub-class: Derived class is a class that inherits from a base class. It is also known as subclass or child class.

Base Class/Superclass: The base class is the main class where derived classes inherit the features. It is also known as the superclass or parent class.

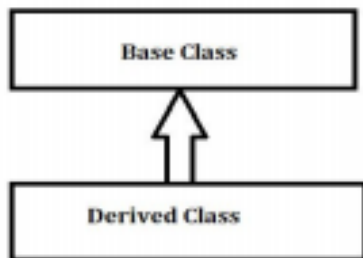
Reusability: The name itself says reuse the repeated code in the programs. It is a mechanism to reuse existing code when you are creating new classes.

The different types of Inheritance are:

- **Single Inheritance**
- **Multiple Inheritance**
- **Multi-Level Inheritance**
- **Hierarchical Inheritance**
- **Hybrid Inheritance**

Single Inheritance

Creating subclasses from a single base class is called single inheritance.



In Inheritance, we can access superclass methods and variables. We can also access subclass methods and variables through subclass objects only. We have to take care of superclass and subclass methods, and variable names shouldn't conflict.

```
class A
{
    int a, b;
    void display()
    {
        System.out.println("Inside class A values =" + a + " " + b);
    }
}
class B extends A
{
    }
```



```
int c;
void show()
{
    System.out.println("Inside Class B values="+a+" "+b+" "+c); }
}
class SingleInheritance
{
    public static void main(String args[])
    {
        B obj = new B(); //derived class object
        obj.a=10;
        obj.b=20;
        obj.c=30;
        obj.display();
        obj.show();
    }
}
```

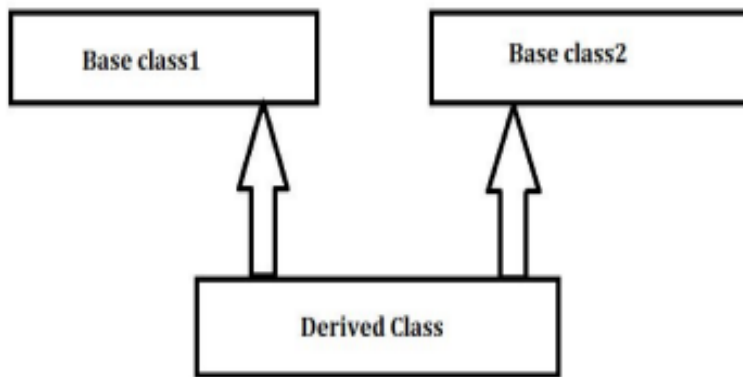
Multiple Inheritance in Java

Defining derived class from numerous base classes is known as 'Multiple Inheritance'. In this case, there is more than one superclass, and there can be one or more subclasses.

Multiple inheritances are available in object-oriented programming with C++, but it is not available in Java.

Java developers want to use multiple inheritances in some cases. Fortunately, Java developers have interface concepts expecting the developers to achieve multiple inheritances by using multiple interfaces.

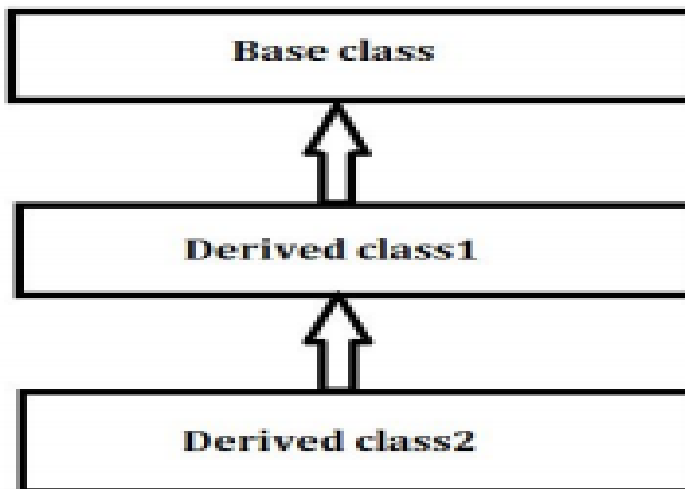
Ex: class Myclass implements interface1, interface2,....



Multi-Level Inheritance in Java

In Multi-Level Inheritance in Java, a class extends to another class that is already extended from another class. For example, if there is a class A that extends class B and class B extends from another class C, then this scenario is known to follow Multi-level Inheritance.

We can take an example of three classes, class Vehicle, class Car, and class SUV. Here, the class Vehicle is the grandfather class. The class Car extends class Vehicle and the class SUV extends class Car.



Hierarchical Inheritance in Java

In Hierarchical Inheritance in Java, more than one derived class extends a single base class. In simple words, more than one child class extends a single parent class or a single parent class has more than one child class.

// Java program to illustrate the

```
// concept of Hierarchical inheritance

class A {
    public void print_A() { System.out.println("Class A"); }
}

class B extends A {
    public void print_B() { System.out.println("Class B"); }
}

class C extends A {
    public void print_C() { System.out.println("Class C"); }
}

class D extends A {
    public void print_D() { System.out.println("Class D"); }
}

// Driver Class
public class Test {
    public static void main(String[] args)
    {
        B obj_B = new B();
        obj_B.print_A();
        obj_B.print_B();

        C obj_C = new C();
        obj_C.print_A();
        obj_C.print_C();

        D obj_D = new D();
        obj_D.print_A();
        obj_D.print_D();
    }
}
```

Hybrid Inheritance(Through Interfaces): It is a mix of two or more of the above types of inheritance. Since java doesn't support multiple inheritances with classes, hybrid

inheritance is also not possible with classes. In java, hybrid inheritance can be achieved only through Interfaces.

Important facts about inheritance in Java

- **Default superclass:** Except Object class, which has no superclass, every class has one and only one direct superclass (single inheritance). In the absence of any other explicit superclass, every class is implicitly a subclass of the Object class.
- **Superclass can only be one:** A superclass can have any number of subclasses. But a subclass can have only **one** superclass. This is because Java does not support multiple inheritances with classes. Although with interfaces, multiple inheritances are supported by java.
- **Inheriting Constructors:** A subclass inherits all the members (fields, methods, and nested classes) from its superclass. Constructors are not members, so they are not inherited by subclasses, but the constructor of the superclass can be invoked from the subclass.
- **Private member inheritance:** A subclass does not inherit the private members of its parent class. However, if the superclass has public or protected methods (like getters and setters) for accessing its private fields, these can also be used by the subclass.

Method Overloading in Java

If a class has multiple methods having same name but different in parameters, it is known as Method Overloading.

If we have to perform only one operation, having same name of the methods increases the readability of the program

Advantage of method overloading

Method overloading increases the readability of the program.

Different ways to overload the method

There are two ways to overload the method in java

- By changing number of arguments
- By changing the data type

1) Method Overloading: changing no. of arguments

In this example, we have created two methods, first add() method performs addition of two numbers and second add method performs addition of three numbers.

```
class Adder{
static int add(int a,int b){return a+b;}
static int add(int a,int b,int c){return a+b+c;}
}
class TestOverloading1 {
public static void main(String[] args){
System.out.println(Adder.add(11,11));
System.out.println(Adder.add(11,11,11));
}}
```

2) Method Overloading: changing data type of arguments

In this example, we have created two methods that differs in data type . The first add method receives two integer arguments and second add method receives two double arguments.

```
class Adder{
static int add(int a, int b){return a+b;}
static double add(double a, double b){return a+b;}
}
class TestOverloading2{
public static void main(String[] args){
System.out.println(Adder.add(11,11));
System.out.println(Adder.add(12.3,12.6));
}}
```

Method Overriding in Java

If subclass (child class) has the same method as declared in the parent class, it is known as method overriding in Java.

In other words, If a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.

Usage of Java Method Overriding

- Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.
- Method overriding is used for runtime polymorphism

Rules for Java Method Overriding

1. The method must have the same name as in the parent class
2. The method must have the same parameter as in the parent class.
3. There must be an IS-A relationship (inheritance).

Example of method overriding

In this example, we have defined the run method in the subclass as defined in the parent class but it has some specific implementation. The name and parameter of the method are the same, and there is IS-A relationship between the classes, so there is method overriding.

//Java Program to illustrate the use of Java Method Overriding

//Creating a parent class.

```
class Vehicle{  
    //defining a method  
    void run(){System.out.println("Vehicle is running");}  
}  
//Creating a child class  
class Bike2 extends Vehicle{  
    //defining the same method as in the parent class  
    void run(){System.out.println("Bike is running safely");}
```

```
    public static void main(String args[]){  
        Bike2 obj = new Bike2();//creating object  
        obj.run();//calling method  
    }  
}
```

//Java Program to demonstrate the real scenario of Java Method Overriding

//where three classes are overriding the method of a parent class.

//Creating a parent class.

```
class Bank{  
    int getRateOfInterest(){return 0;}  
}
```

//Creating child classes.

```
class SBI extends Bank{
int getRateOfInterest(){return 8;}
}

class ICICI extends Bank{
int getRateOfInterest(){return 7;}
}
class AXIS extends Bank{
int getRateOfInterest(){return 9;}
}
//Test class to create objects and call the methods
class Test2{
public static void main(String args[]){
SBI s=new SBI();
ICICI i=new ICICI();
AXIS a=new AXIS();
System.out.println("SBI Rate of Interest: "+s.getRateOfInterest());
System.out.println("ICICI Rate of Interest: "+i.getRateOfInterest());
System.out.println("AXIS Rate of Interest: "+a.getRateOfInterest());
}
}
```

Super Keyword in Java

The **super** keyword in Java is a reference variable which is used to refer immediate parent class object.

Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by super reference variable.

Usage of Java super Keyword

1. super can be used to refer immediate parent class instance variable.
2. super can be used to invoke immediate parent class method.
3. super() can be used to invoke immediate parent class constructor.

1) super is used to refer immediate parent class instance variable.

We can use super keyword to access the data member or field of parent class. It is used if parent class and child class have same fields.

```
class Animal{
String color="white";
}
```

```
class Dog extends Animal{
String color="black";
void printColor(){
System.out.println(color);//prints color of Dog class
System.out.println(super.color);//prints color of Animal class
}
}
class TestSuper1 {
public static void main(String args[]){
Dog d=new Dog();
d.printColor();
}}
```

2) super can be used to invoke parent class method

The super keyword can also be used to invoke parent class method. It should be used if subclass contains the same method as parent class. In other words, it is used if method is overridden.

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void eat(){System.out.println("eating bread...");}
void bark(){System.out.println("barking...");}
void work(){
super.eat();
bark();
}
}
class TestSuper2{
public static void main(String args[]){
Dog d=new Dog();
d.work();
}}
```

3) super is used to invoke parent class constructor.

The super keyword can also be used to invoke the parent class constructor. Let's see a simple example:

```
class Animal{
```



```
Animal(){System.out.println("animal is created");}
}
class Dog extends Animal{
Dog(){
super();
System.out.println("dog is created");
}
}
class TestSuper3{
public static void main(String args[]){
Dog d=new Dog();
}}
```

Instance initializer block

Instance Initializer block is used to initialize the instance data member. It runs each time when an object of the class is created.

The initialization of the instance variable can be done directly but there can be performed extra operations while initializing the instance variable in the instance initializer block.

Rules for instance initializer block :

There are mainly three rules for the instance initializer block. They are as follows:

1. The instance initializer block is created when an instance of the class is created.
2. The instance initializer block is invoked after the parent class constructor is invoked (i.e. after `super()` constructor call).
3. The instance initializer block comes in the order in which they appear.

Final Keyword In Java

The **final keyword** in java is used to restrict the user. The java final keyword can be used in many contexts. Final can be:

1. variable
2. method
3. class

The final keyword can be applied with the variables, a final variable that has no value is called a blank final variable or uninitialized final variable. It can be initialized in the constructor only. The blank final variable can be static also which will be initialized in

the static block only. We will have detailed learning of these. Let's first learn the basics of final keyword.

1) Java final variable

If you make any variable as final, you cannot change the value of final variable(It will be constant).

Example of final variable

There is a final variable speedlimit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

```
class Bike9{
    final int speedlimit=90;//final variable
    void run(){
        speedlimit=400;
    }
    public static void main(String args[]){
        Bike9 obj=new Bike9();
        obj.run();
    }
} //end of class
```

2) Java final method

If you make any method as final, you cannot override it.

Example of final method

```
class Bike{
    final void run(){System.out.println("running");}
}

class Honda extends Bike{
    void run(){System.out.println("running safely with 100kmph");}

    public static void main(String args[]){
        Honda honda= new Honda();
        honda.run();
    }
}
```

3) Java final class

If you make any class as final, you cannot extend it.

Example of final class

```
final class Bike{}

class Honda1 extends Bike{

    void run(){System.out.println("running safely with 100kmph");}

    public static void main(String args[]){

        Honda1 honda= new Honda1();

        honda.run();

    }

}
```

Polymorphism in Java

Polymorphism in Java is a concept by which we can perform a *single action in different ways*. Polymorphism is derived from 2 Greek words: poly and morphs. The word "poly" means many and "morphs" means forms. So polymorphism means many forms.

There are two types of polymorphism in Java: compile-time polymorphism and runtime polymorphism. We can perform polymorphism in java by method overloading and method overriding.

Runtime Polymorphism in Java

Runtime polymorphism or Dynamic Method Dispatch is a process in which a call to an overridden method is resolved at runtime rather than compile-time.

Example of Java Runtime Polymorphism

In this example, we are creating two classes Bike and Splendor. Splendor class extends Bike class and overrides its run() method. We are calling the run method by the

reference variable of Parent class. Since it refers to the subclass object and subclass method overrides the Parent class method, the subclass method is invoked at runtime.

```
class Bike{  
    void run(){System.out.println("running");}  
}  
  
class Splendor extends Bike{  
    void run(){System.out.println("running safely with 60km");}  
  
    public static void main(String args[]){  
        Bike b = new Splendor();//upcasting  
        b.run();  
    }  
}
```

Static Binding and Dynamic Binding

Connecting a method call to the method body is known as binding.

There are two types of binding

1. Static Binding (also known as Early Binding).
2. Dynamic Binding (also known as Late Binding).

static binding

When type of the object is determined at compiled time(by the compiler), it is known as static binding.

If there is any private, final or static method in a class, there is static binding.

Example of static binding

```
class Dog{  
    private void eat(){System.out.println("dog is eating...");}
```

```
public static void main(String args[]){  
    Dog d1=new Dog();  
    d1.eat();  
}  
}
```

Dynamic binding

When type of the object is determined at run-time, it is known as dynamic binding.

```
class Animal{  
    void eat(){System.out.println("animal is eating...");}  
}
```

```
class Dog extends Animal{  
    void eat(){System.out.println("dog is eating...");}
```

```
public static void main(String args[]){  
    Animal a=new Dog();  
    a.eat();  
}  
}
```

Java instanceof

The java instanceof operator is used to test whether the object is an instance of the specified type (class or subclass or interface).

The instanceof in java is also known as type comparison operator because it compares the instance with type. It returns either true or false. If we apply the instanceof operator with any variable that has null value, it returns false.

```
class Simple1 {
```

```
public static void main(String args[]){  
    Simple1 s=new Simple1();  
    System.out.println(s instanceof Simple1);//true  
}  
}
```

Abstract class in Java

A class which is declared with the abstract keyword is known as an abstract class in Java. It can have abstract and non-abstract methods (method with the body).

Abstraction in Java

Abstraction is a process of hiding the implementation details and showing only functionality to the user.

Another way, it shows only essential things to the user and hides the internal details, for example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.

Abstract class in Java

A class which is declared as abstract is known as an **abstract class**. It can have abstract and non-abstract methods. It needs to be extended and its method implemented. It cannot be instantiated.

Points to Remember

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have [constructors](#) and static methods also.
- It can have final methods which will force the subclass not to change the body of the method.

Example of abstract class

1. **abstract class A{}**

Abstract Method in Java

A method which is declared as abstract and does not have implementation is known as an abstract method.

Example of abstract method

```
abstract void printStatus();//no method body and abstract
```

Example of Abstract class that has an abstract method

In this example, Bike is an abstract class that contains only one abstract method run. Its implementation is provided by the Honda class.

```
abstract class Bike{
    abstract void run();
}
class Honda4 extends Bike{
    void run(){System.out.println("running safely");}
    public static void main(String args[]){
        Bike obj = new Honda4();
        obj.run();
    }
}
```

Interface in Java

An **interface in Java** is a blueprint of a class. It has static constants and abstract methods.

The interface in Java is *a mechanism to achieve abstraction*. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritance in Java.

In other words, you can say that interfaces can have abstract methods and variables. It cannot have a method body.

Java Interface also **represents the IS-A relationship**.

There are mainly three reasons to use interface. They are given below.

- It is used to achieve abstraction.

- By interface, we can support the functionality of multiple inheritance.
- It can be used to achieve loose coupling.

to declare an interface

An interface is declared by using the interface keyword. It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default. A class that implements an interface must implement all the methods declared in the interface.

Syntax:

```
interface <interface_name>{  
  
    // declare constant fields  
    // declare methods that abstract  
    // by default.  
}
```

Example

```
//Interface declaration: by first user  
interface Drawable{  
    void draw();  
}  
//Implementation: by second user  
class Rectangle implements Drawable{  
    public void draw(){System.out.println("drawing rectangle");}  
}  
class Circle implements Drawable{  
    public void draw(){System.out.println("drawing circle");}  
}  
//Using interface: by third user  
class TestInterface1 {  
    public static void main(String args[]){  
        Drawable d=new Circle();//In real scenario, object is provided by method e.g. getDrawable()  
        d.draw();  
    }  
}
```

Difference between abstract class and interface

Abstract class and interface both are used to achieve abstraction where we can declare the abstract methods. Abstract class and interface both can't be instantiated. But there are many differences between abstract class and interface that are given below.

Abstract class	Interface
1) Abstract class can have abstract and non-abstract methods.	Interface can have only abstract methods. Since Java 8, it can have default and static methods also.
2) Abstract class doesn't support multiple inheritance.	Interface supports multiple inheritance.
3) Abstract class can have final, non-final, static and non-static variables.	Interface has only static and final variables.
4) Abstract class can provide the implementation of interface.	Interface can't provide the implementation of abstract class.
5) The abstract keyword is used to declare abstract class.	The interface keyword is used to declare interface.
6) An abstract class can extend another Java class and implement multiple Java interfaces.	An interface can extend another Java interface only.
7) An abstract class can be extended using keyword "extends".	An interface can be implemented using keyword "implements".
8) A Java abstract class can have class members like private, protected, etc.	Members of a Java interface are public by default.
9)Example: <pre>public abstract class Shape{ public abstract void draw(); }</pre>	Example: <pre>public interface Drawable{ void draw(); }</pre>

```
//Creating interface that has 4 methods
interface A{
```

```
void a();//bydefault, public and abstract
void b();
void c();
void d();
}
```

```
//Creating abstract class that provides the implementation of one method of A interface
abstract class B implements A{
public void c(){System.out.println("I am C");}
}
```

```
//Creating subclass of abstract class, now we need to provide the implementation of rest
of the methods
class M extends B{
public void a(){System.out.println("I am a");}
public void b(){System.out.println("I am b");}
public void d(){System.out.println("I am d");}
}
```

```
//Creating a test class that calls the methods of A interface
class Test5{
public static void main(String args[]){
A a=new M();
a.a();
a.b();
a.c();
a.d();
}}
```

Java Package

A **java package** is a group of similar types of classes, interfaces and sub-packages. Package in java can be categorized in two form, built-in package and user-defined package.

There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.

Advantage of Java Package

1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.

- 2) Java package provides access protection.
- 3) Java package removes naming collision.

How to access package from another package

There are three ways to access the package from outside the package.

- 1. `import package.*;`
- 2. `import package.classname;`
- 3. fully qualified name.

1) Using packagename.*

If you use `package.*` then all the classes and interfaces of this package will be accessible but not subpackages.

The `import` keyword is used to make the classes and interface of another package accessible to the current package.

Example of package that import the packagename.*

```
//save by A.java
package pack;
public class A{
    public void msg(){System.out.println("Hello");}
}

//save by B.java
package mypack;
import pack.*;

class B{
    public static void main(String args[]){
        A obj = new A();
        obj.msg();
    }
}
```

2) Using packagename.classname

If you import `package.classname` then only declared class of this package will be accessible.

Example of package by import `package.classname`

```
//save by A.java
```

```
package pack;
public class A{
    public void msg(){System.out.println("Hello");}
}
//save by B.java
package mypack;
import pack.A;

class B{
    public static void main(String args[]){
        A obj = new A();
        obj.msg();
    }
}
```

3) Using fully qualified name

If you use fully qualified name then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.

It is generally used when two packages have same class name e.g. java.util and java.sql packages contain Date class.

Example of package by import fully qualified name

```
//save by A.java
package pack;
public class A{
    public void msg(){System.out.println("Hello");}
}
//save by B.java
package mypack;
class B{
    public static void main(String args[]){
        pack.A obj = new pack.A();//using fully qualified name
        obj.msg();
    }
}
```

Access Modifiers in Java

There are two types of modifiers in Java: access modifiers and non-access modifiers.

The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class. We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

There are four types of Java access modifiers:

Private: The access level of a private modifier is only within the class. It cannot be accessed from outside the class.

Default: The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.

Protected: The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.

Public: The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

There are many non-access modifiers, such as static, abstract, synchronized, native, volatile, transient, etc. Here, we are going to learn the access modifiers only.

1) Private

The private access modifier is accessible only within the class.

Simple example of private access modifier

In this example, we have created two classes A and Simple. A class contains private data member and private method. We are accessing these private members from outside the class, so there is a compile-time error.

```
class A{  
private int data=40;  
private void msg(){System.out.println("Hello java");}  
}
```

```
public class Simple{
```

```
public static void main(String args[]){
    A obj=new A();
    System.out.println(obj.data);//Compile Time Error
    obj.msg();//Compile Time Error
}
}
```

Role of Private Constructor

If you make any class constructor private, you cannot create the instance of that class from outside the class. For example:

```
class A{
private A(){}//private constructor
void msg(){System.out.println("Hello java");}
}
public class Simple{
    public static void main(String args[]){
        A obj=new A();//Compile Time Error
    }
}
```

2) Default

If you don't use any modifier, it is treated as default by default. The default modifier is accessible only within package. It cannot be accessed from outside the package. It provides more accessibility than private. But, it is more restrictive than protected, and public.

Example of default access modifier

In this example, we have created two packages pack and mypack. We are accessing the A class from outside its package, since A class is not public, so it cannot be accessed from outside the package

```
//save by A.java
package pack;
class A{
    void msg(){System.out.println("Hello");}
}
//save by B.java
package mypack;
```

```
import pack.*;
class B{
    public static void main(String args[]){
        A obj = new A();//Compile Time Error
        obj.msg();//Compile Time Error
    }
}
```

3) Protected

The protected access modifier is accessible within package and outside the package but through inheritance only.

The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

It provides more accessibility than the default modifier.

Example of protected access modifier

In this example, we have created the two packages pack and mypack. The A class of pack package is public, so can be accessed from outside the package. But msg method of this package is declared as protected, so it can be accessed from outside the class only through inheritance.

```
//save by A.java
package pack;
public class A{
    protected void msg(){System.out.println("Hello");}
}
//save by B.java
package mypack;
import pack.*;

class B extends A{
    public static void main(String args[]){
        B obj = new B();
        obj.msg();
    }
}
```

4) Public

The public access modifier is accessible everywhere. It has the widest scope among all other modifiers.

Example of public access modifier

//save by A.java

```
package pack;
public class A {
    public void msg(){System.out.println("Hello");}
}
```

//save by B.java

```
package mypack;
import pack.*;

class B {
    public static void main(String args[]){
        A obj = new A();
        obj.msg();
    }
}
```

Encapsulation in Java

Encapsulation in Java is a *process of wrapping code and data together into a single unit*, for example, a capsule which is mixed of several medicines.

A fully encapsulated class in Java can be created by making all the data members of the class private. Now we can use setter and getter methods to set and get the data in it.

The Java Bean class is the example of a fully encapsulated class.

Advantage of Encapsulation in Java

By providing only a setter or getter method, you can make the class read-only or write-only. In other words, you can skip the getter or setter methods.

It provides you the control over the data. Suppose you want to set the value of id which should be greater than 100 only, you can write the logic inside the setter method. You can write the logic not to store the negative numbers in the setter methods.

It is a way to achieve data hiding in Java because other class will not be able to access the data through the private data members.

The encapsulate class is easy to test. So, it is better for unit testing.

The standard IDE's are providing the facility to generate the getters and setters. So, it is easy and fast to create an encapsulated class in Java.

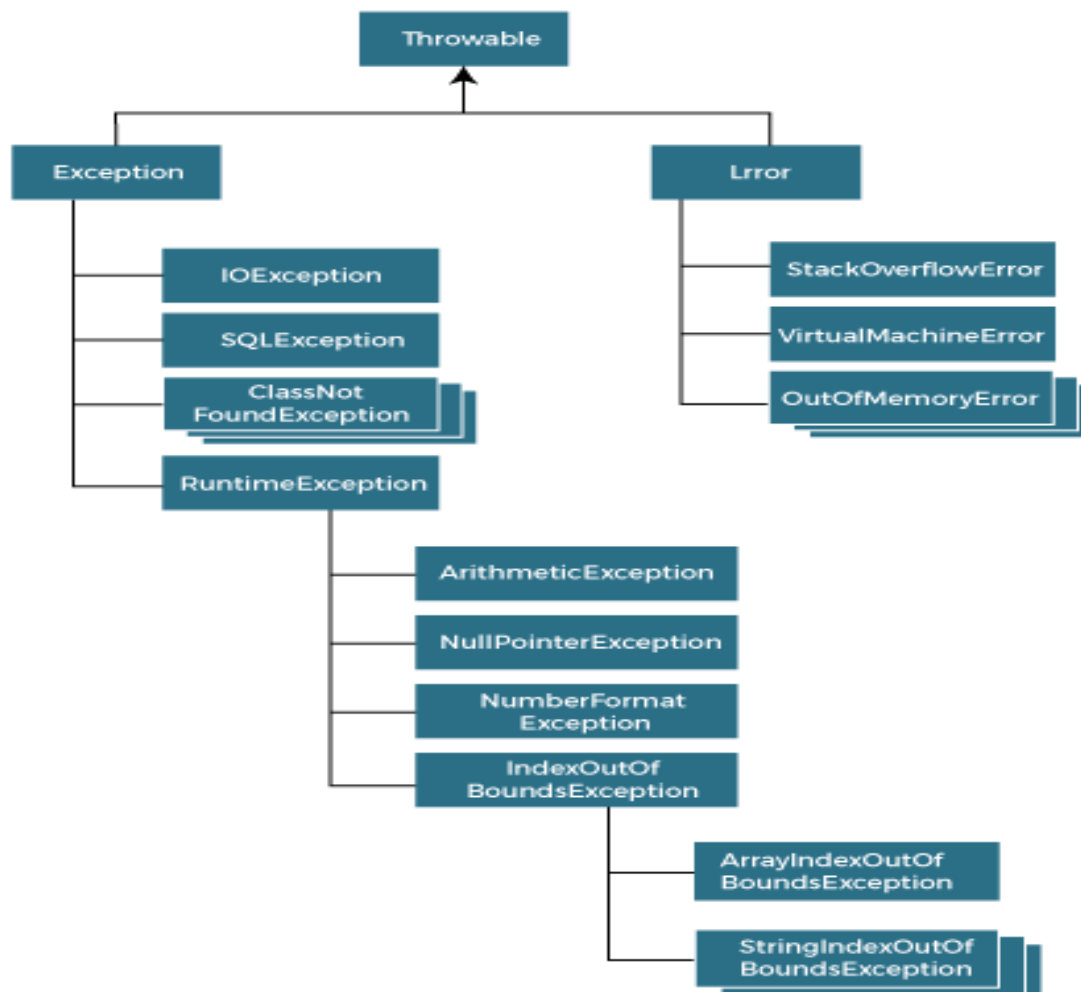
Module III: Exception Handling, The Try-Catch Statement, Catching more than one Exception, The Finally Clause, Generating Exceptions, Threads: Introduction, Creating Threads in Applications, Method in Thread Class, Threads in Applets.

Exception Handling in Java

The Exception Handling in Java is one of the powerful mechanism to handle the runtime errors so that the normal flow of the application can be maintained. In Java, an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime. Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IOException, SQLException, RemoteException, etc. The core advantage of exception handling is to maintain the normal flow of the application. An exception normally disrupts the normal flow of the application.

Hierarchy of Java Exception classes

The java.lang.Throwable class is the root class of Java Exception hierarchy inherited by two subclasses: Exception and Error. The hierarchy of Java Exception classes is given below:



Types of Java Exceptions

There are mainly two types of exceptions: checked and unchecked. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:

1. Checked Exception
2. Unchecked Exception
3. Error

Difference between Checked and Unchecked Exceptions

1) Checked Exception

The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

3) Error

Error is irrecoverable. Some example of errors are OutOfMemoryError, VirtualMachineError, AssertionError etc.

Java Exception Keywords

Java provides five keywords that are used to handle the exception. The following table describes each.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed

	by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

Java Exception Handling Example

```
public class JavaExceptionExample{
    public static void main(String args[]){
        try{
            //code that may raise exception
            int data=100/0;
        }catch(ArithmeticException e){System.out.println(e);}
        //rest code of the program
        System.out.println("rest of the code...");
    }
}
```

Common Scenarios of Java Exceptions

There are given some scenarios where unchecked exceptions may occur. They are as follows:

1) A scenario where ArithmeticException occurs

If we divide any number by zero, there occurs an ArithmeticException.

int a=50/0;//ArithmeticException

2) A scenario where NullPointerException occurs

If we have a null value in any variable, performing any operation on the variable throws a NullPointerException.

String s=null;

System.out.println(s.length());//NullPointerException

3) A scenario where NumberFormatException occurs

If the formatting of any variable or number is mismatched, it may result into `NumberFormatException`. Suppose we have a string variable that has characters; converting this variable into digit will cause `NumberFormatException`.

```
String s="abc";
```

```
int i=Integer.parseInt(s);//NumberFormatException
```

4) A scenario where `ArrayIndexOutOfBoundsException` occurs

When an array exceeds to its size, the `ArrayIndexOutOfBoundsException` occurs. There may be other reasons to occur `ArrayIndexOutOfBoundsException`. Consider the following statements.

```
int a[]=new int[5];
```

```
a[10]=50; //ArrayIndexOutOfBoundsException
```

Java try block

Java try block is used to enclose the code that might throw an exception. It must be used within the method.

If an exception occurs at the particular statement in the try block, the rest of the block code will not execute.

Java try block must be followed by either catch or finally block.

Syntax of Java try-catch

```
try{  
//code that may throw an exception  
}catch(Exception_class_Name ref){}
```

Syntax of try-finally block

```
try{  
//code that may throw an exception  
}finally{}
```

Java catch block

Java catch block is used to handle the Exception by declaring the type of exception within the parameter. The declared exception must be the parent class exception (i.e., `Exception`) or the generated exception type.

The JVM firstly checks whether the exception is handled or not. If exception is not handled, JVM provides a default exception handler that performs the following tasks:

- Prints out exception description.
- Prints the stack trace (Hierarchy of methods where the exception occurred).
- Causes the program to terminate.

But if the application programmer handles the exception, the normal flow of the application is maintained, i.e., rest of the code is executed.

Solution by exception handling

```
public class TryCatchExample {  
  
    public static void main(String[] args) {  
        try  
        {  
            int data=50/0; //may throw exception  
        }  
        //handling the exception  
        catch(ArithmeticException e)  
        {  
            System.out.println(e);  
        }  
        System.out.println("rest of the code");  
    }  
  
}
```

TryCatchExample3.java

```
public class TryCatchExample3 {  
  
    public static void main(String[] args) {  
        try  
        {  
            int data=50/0; //may throw exception  
            // if exception occurs, the remaining statement will not execute  
            System.out.println("rest of the code");  
        }  
        // handling the exception  
        catch(ArithmeticException e)
```

```
        {  
            System.out.println(e);  
        }  
  
    }  
  
}
```

if an exception occurs in the try block, the rest of the block code will not execute.

example to resolve the exception in a catch block.

```
public class TryCatchExample6 {  
  
    public static void main(String[] args) {  
        int i=50;  
        int j=0;  
        int data;  
        try  
        {  
            data=i/j; //may throw exception  
        }  
        // handling the exception  
        catch(Exception e)  
        {  
            // resolving the exception in catch block  
            System.out.println(i/(j+2));  
        }  
    }  
}
```

Java Multi-catch block

```
public class MultipleCatchBlock1 {  
  
    public static void main(String[] args) {  
  
        try{  
            int a[]=new int[5];  
            a[5]=30/0;  
        }
```

```
        catch(ArithmeticException e)
        {
            System.out.println("Arithmetic Exception occurs");
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println("ArrayIndexOutOfBoundsException Exception occurs");
        }
        catch(Exception e)
        {
            System.out.println("Parent Exception occurs");
        }
        System.out.println("rest of the code");
    }
}
```

Example 2

```
public class MultipleCatchBlock2 {

    public static void main(String[] args) {

        try{
            int a[]=new int[5];

            System.out.println(a[10]);
        }
        catch(ArithmeticException e)
        {
            System.out.println("Arithmetic Exception occurs");
        }
        catch(ArrayIndexOutOfBoundsException e)
        {
            System.out.println("ArrayIndexOutOfBoundsException Exception occurs");
        }
        catch(Exception e)
        {
            System.out.println("Parent Exception occurs");
        }
    }
}
```



```
        System.out.println("rest of the code");
    }
}
```

In this example, try block contains two exceptions. But at a time only one exception occurs and its corresponding catch block is executed.

Java Nested try block

In Java, using a try block inside another try block is permitted. It is called as nested try block. Every statement that we enter a statement in try block, context of that exception is pushed onto the stack.

For example, the inner try block can be used to handle `ArrayIndexOutOfBoundsException` while the outer try block can handle the `ArithmeticException` (division by zero).

```
//main try block
try
{
    statement 1;
    statement 2;
    //try catch block within another try block
    try
    {
        statement 3;
        statement 4;
        //try catch block within nested try block
        try
        {
            statement 5;
            statement 6;
        }
        catch(Exception e2)
        {
            //exception message
        }

    }
    catch(Exception e1)
    {
        //exception message
    }
}
```

```
    }  
}  
//catch block of parent (outer) try block  
catch(Exception e3)  
{  
    //exception message  
}  
....
```

an example where we place a try block within another try block for two different exceptions.

```
public class NestedTryBlock{  
    public static void main(String args[]){  
        //outer try block  
        try{  
            //inner try block 1  
            try{  
                System.out.println("going to divide by 0");  
                int b =39/0;  
            }  
            //catch block of inner try block 1  
            catch(ArithmeticException e)  
            {  
                System.out.println(e);  
            }  
  
            //inner try block 2  
            try{  
                int a[]=new int[5];  
  
                //assigning the value out of array bounds  
                a[5]=4;  
            }  
  
            //catch block of inner try block 2  
            catch(ArrayIndexOutOfBoundsException e)  
            {
```

```
        System.out.println(e);
    }

    System.out.println("other statement");
}
//catch block of outer try block
catch(Exception e)
{
    System.out.println("handled the exception (outer catch)");
}

System.out.println("normal flow..");
}
}
```

Here the try block within nested try block (inner try block 2) do not handle the exception. The control is then transferred to its parent try block (inner try block 1). If it does not handle the exception, then the control is transferred to the main try block (outer try block) where the appropriate catch block handles the exception. It is termed as nesting.

NestedTryBlock.java

```
public class NestedTryBlock2 {

    public static void main(String args[])
    {
        // outer (main) try block
        try {

            //inner try block 1
            try {

                // inner try block 2
                try {
                    int arr[] = { 1, 2, 3, 4 };

                    //printing the array element out of its bounds
```

```
        System.out.println(arr[10]);
    }

    // to handles ArithmeticException
    catch (ArithmeticException e) {
        System.out.println("Arithmetic exception");
        System.out.println(" inner try block 2");
    }
}

// to handle ArithmeticException
catch (ArithmeticException e) {
    System.out.println("Arithmetic exception");
    System.out.println("inner try block 1");
}
}

// to handle ArrayIndexOutOfBoundsException
catch (ArrayIndexOutOfBoundsException e4) {
    System.out.print(e4);
    System.out.println(" outer (main) try block");
}
catch (Exception e5) {
    System.out.print("Exception");
    System.out.println(" handled in main try-block");
}
}
}
```

Java finally block

Java finally block is a block used to execute important code such as closing the connection, etc.

Java finally block is always executed whether an exception is handled or not. Therefore, it contains all the necessary statements that need to be printed regardless of the exception occurs or not.

The finally block follows the try-catch block.

```
class TestFinallyBlock {
    public static void main(String args[]){
        try{
//below code do not throw any exception
            int data=25/5;
            System.out.println(data);
        }
//catch won't be executed
        catch(NullPointerException e){
System.out.println(e);
        }
//executed regardless of exception occurred or not
        finally {
System.out.println("finally block is always executed");
        }

System.out.println("rest of phe code...");
    }
}
```

When an exception occurs but not handled by the catch block

Let's see the following example. Here, the code throws an exception however the catch block cannot handle it. Despite this, the finally block is executed after the try block and then the program terminates abnormally.

```
public class TestFinallyBlock1 {
    public static void main(String args[]){

        try {

            System.out.println("Inside the try block");

            //below code throws divide by zero exception
            int data=25/0;
            System.out.println(data);
        }
//cannot handle Arithmetic type exception
```

```
//can only accept Null Pointer type exception
catch(NullPointerException e){
    System.out.println(e);
}

//executes regardless of exception occurred or not
finally {
    System.out.println("finally block is always executed");
}

System.out.println("rest of the code...");
}
}
```

Case 3: When an exception occurs and is handled by the catch block

```
public class TestFinallyBlock2{
    public static void main(String args[]){

        try {

            System.out.println("Inside try block");

            //below code throws divide by zero exception
            int data=25/0;
            System.out.println(data);
        }

        //handles the Arithmetic Exception / Divide by zero exception
        catch(ArithmeticException e){
            System.out.println("Exception handled");
            System.out.println(e);
        }

        //executes regardless of exception occurred or not
        finally {
            System.out.println("finally block is always executed");
        }
    }
}
```

```
    System.out.println("rest of the code...");  
}  
}
```

Java throw keyword

The Java throw keyword is used to throw an exception explicitly.

We specify the exception object which is to be thrown. The Exception has some message with it that provides the error description. These exceptions may be related to user inputs, server, etc.

The syntax of the Java throw keyword is given below.

1. **throw new** exception_class("error message");
2. throw new IOException("sorry device error");

TestThrow1.java

the validate method that takes integer value as a parameter. If the age is less than 18, we are throwing the ArithmeticException otherwise print a message welcome to vote.

```
public class TestThrow1 {  
    //function to check if person is eligible to vote or not  
    public static void validate(int age) {  
        if(age<18) {  
            //throw Arithmetic exception if not eligible to vote  
            throw new ArithmeticException("Person is not eligible to vote");  
        }  
        else {  
            System.out.println("Person is eligible to vote!!");  
        }  
    }  
    //main method  
    public static void main(String args[]){  
        //calling the function  
        validate(13);  
        System.out.println("rest of the code...");  
    }  
}
```

Java Thread class

Java provides Thread class to achieve thread programming. Thread class provides constructors and methods to create and perform operations on a thread. Thread class extends Object class and implements Runnable interface.

Java Thread Methods

S.N .	Modifier and Type	Method	Description
1)	void	start()	It is used to start the execution of the thread.
2)	void	run()	It is used to do an action for a thread.
3)	static void	sleep()	It sleeps a thread for the specified amount of time.
4)	static Thread	currentThread()	It returns a reference to the currently executing thread object.
5)	void	join()	It waits for a thread to die.
6)	int	getPriority()	It returns the priority of the thread.
7)	void	setPriority()	It changes the priority of the thread.
8)	String	getName()	It returns the name of the thread.
9)	void	setName()	It changes the name of the thread.
10)	long	getId()	It returns the id of the thread.
11)	boolean	isAlive()	It tests if the thread is alive.

12)	static void	yield()	It causes the currently executing thread object to pause and allow other threads to execute temporarily.
13)	void	suspend()	It is used to suspend the thread.
14)	void	resume()	It is used to resume the suspended thread.
15)	void	stop()	It is used to stop the thread.
16)	void	destroy()	It is used to destroy the thread group and all of its subgroups.
17)	boolean	isDaemon()	It tests if the thread is a daemon thread.
18)	void	setDaemon()	It marks the thread as daemon or user thread.
19)	void	interrupt()	It interrupts the thread.
20)	boolean	isinterrupted()	It tests whether the thread has been interrupted.
21)	static boolean	interrupted()	It tests whether the current thread has been interrupted.
22)	static int	activeCount()	It returns the number of active threads in the current thread's thread group.
23)	void	checkAccess()	It determines if the currently running thread has permission to modify the thread.

24)	static boolean	<u>holdLock()</u>	It returns true if and only if the current thread holds the monitor lock on the specified object.
25)	static void	<u>dumpStack()</u>	It is used to print a stack trace of the current thread to the standard error stream.
26)	StackTraceElement[]	<u>getStackTrace()</u>	It returns an array of stack trace elements representing the stack dump of the thread.
27)	static int	enumerate()	It is used to copy every active thread's thread group and its subgroup into the specified array.
28)	Thread.State	getState()	It is used to return the state of the thread.
29)	ThreadGroup	getThreadGroup()	It is used to return the thread group to which this thread belongs
30)	String	toString()	It is used to return a string representation of this thread, including the thread's name, priority, and thread group.
31)	void	notify()	It is used to give the notification for only one thread which is waiting for a particular object.
32)	void	notifyAll()	It is used to give the notification to all waiting threads of a particular object.

33)	void	setContextClassLoader()	It sets the context ClassLoader for the Thread.
34)	ClassLoader	getContextClassLoader()	It returns the context ClassLoader for the thread.
35)	static Thread.Uncaught ExceptionHandler	getDefaultUncaughtExceptionHandler()	It returns the default handler invoked when a thread abruptly terminates due to an uncaught exception.
36)	static void	setDefaultUncaughtExceptionHandler()	It sets the default handler invoked when a thread abruptly terminates due to an uncaught exception.

Life cycle of a Thread (Thread States)

In Java, a thread always exists in any one of the following states. These states are:

1. New
2. Active
3. Blocked / Waiting
4. Timed Waiting
5. Terminated

Explanation of Different Thread States

New: Whenever a new thread is created, it is always in the new state. For a thread in the new state, the code has not been run yet and thus has not begun its execution.

Active: When a thread invokes the start() method, it moves from the new state to the active state. The active state contains two states within it: one is runnable, and the other is running.

Runnable: A thread, that is ready to run is then moved to the runnable state. In the runnable state, the thread may be running or may be ready to run at any given instant of time. It is the duty of the thread scheduler to provide the thread time to run, i.e., moving the thread the running state.

A program implementing multithreading acquires a fixed slice of time to each individual thread. Each and every thread runs for a short span of time and when that allocated time

slice is over, the thread voluntarily gives up the CPU to the other thread, so that the other threads can also run for their slice of time. Whenever such a scenario occurs, all those threads that are willing to run, waiting for their turn to run, lie in the runnable state. In the runnable state, there is a queue where the threads lie.

Running: When the thread gets the CPU, it moves from the runnable to the running state. Generally, the most common change in the state of a thread is from runnable to running and again back to runnable.

Blocked or Waiting: Whenever a thread is inactive for a span of time (not permanently) then, either the thread is in the blocked state or is in the waiting state.

For example, a thread (let's say its name is A) may want to print some data from the printer. However, at the same time, the other thread (let's say its name is B) is using the printer to print some data. Therefore, thread A has to wait for thread B to use the printer. Thus, thread A is in the blocked state. A thread in the blocked state is unable to perform any execution and thus never consume any cycle of the Central Processing Unit (CPU). Hence, we can say that thread A remains idle until the thread scheduler reactivates thread A, which is in the waiting or blocked state.

When the main thread invokes the `join()` method then, it is said that the main thread is in the waiting state. The main thread then waits for the child threads to complete their tasks. When the child threads complete their job, a notification is sent to the main thread, which again moves the thread from waiting to the active state.

If there are a lot of threads in the waiting or blocked state, then it is the duty of the thread scheduler to determine which thread to choose and which one to reject, and the chosen thread is then given the opportunity to run.

Timed Waiting: Sometimes, waiting for leads to starvation. For example, a thread (its name is A) has entered the critical section of a code and is not willing to leave that critical section. In such a scenario, another thread (its name is B) has to wait forever, which leads to starvation. To avoid such scenario, a timed waiting state is given to thread B. Thus, thread lies in the waiting state for a specific span of time, and not forever. A real example of timed waiting is when we invoke the `sleep()` method on a specific thread. The `sleep()` method puts the thread in the timed wait state. After the time runs out, the thread wakes up and start its execution from when it has left earlier.

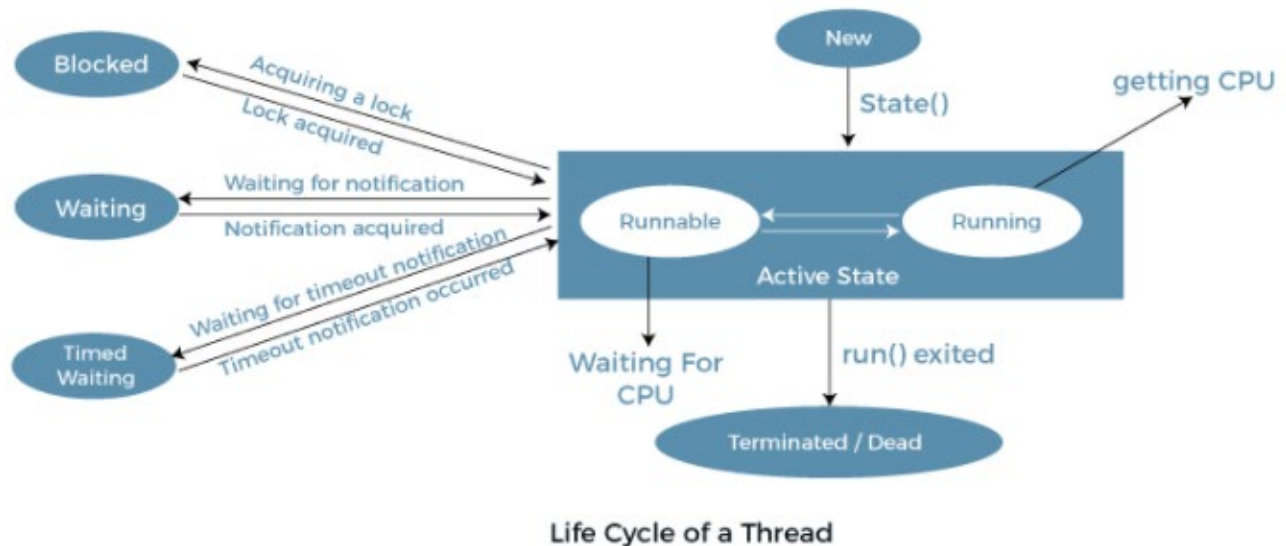
Terminated: A thread reaches the termination state because of the following reasons:

When a thread has finished its job, then it exists or terminates normally.

Abnormal termination: It occurs when some unusual events such as an unhandled exception or segmentation fault.

A terminated thread means the thread is no more in the system. In other words, the thread is dead, and there is no way one can respawn (active after kill) the dead thread.

The following diagram shows the different states involved in the life cycle of a thread.



Implementation of Thread States

In Java, one can get the current state of a thread using the `Thread.getState()` method. The `java.lang.Thread.State` class of Java provides the constants ENUM to represent the state of a thread. These constants are:

1. **public static final Thread.State NEW**

It represents the first state of a thread that is the NEW state.

2. **public static final Thread.State RUNNABLE**

It represents the runnable state. It means a thread is waiting in the queue to run.

3. **public static final Thread.State BLOCKED**

It represents the blocked state. In this state, the thread is waiting to acquire a lock.

4. **public static final Thread.State WAITING**

It represents the waiting state. A thread will go to this state when it invokes the `Object.wait()` method, or `Thread.join()` method with no timeout. A thread in the waiting state is waiting for another thread to complete its task.

5. **public static final** Thread.State TIMED_WAITING

It represents the timed waiting state. The main difference between waiting and timed waiting is the time constraint. Waiting has no time constraint, whereas timed waiting has the time constraint. A thread invoking the following method reaches the timed waiting state.

- `sleep`
- `join` with timeout
- `wait` with timeout
- `parkUntil`
- `parkNanos`

Java Threads | How to create a thread in Java

There are two ways to create a thread:

1. By extending Thread class
2. By implementing Runnable interface.

Commonly used Constructors of Thread class:

- `Thread()`
- `Thread(String name)`
- `Thread(Runnable r)`
- `Thread(Runnable r,String name)`

Commonly used methods of Thread class:

1. **public void run():** is used to perform action for a thread.
2. **public void start():** starts the execution of the thread.JVM calls the `run()` method on the thread.
3. **public void sleep(long milliseconds):** Causes the currently executing thread to sleep (temporarily cease execution) for the specified number of milliseconds.
4. **public void join():** waits for a thread to die.
5. **public void join(long milliseconds):** waits for a thread to die for the specified milliseconds.
6. **public int getPriority():** returns the priority of the thread.
7. **public int setPriority(int priority):** changes the priority of the thread.
8. **public String getName():** returns the name of the thread.
9. **public void setName(String name):** changes the name of the thread.

10. **public Thread currentThread():** returns the reference of currently executing thread.
11. **public int getId():** returns the id of the thread.
12. **public Thread.State getState():** returns the state of the thread.
13. **public boolean isAlive():** tests if the thread is alive.
14. **public void yield():** causes the currently executing thread object to temporarily pause and allow other threads to execute.
15. **public void suspend():** is used to suspend the thread(deprecated).
16. **public void resume():** is used to resume the suspended thread(deprecated).
17. **public void stop():** is used to stop the thread(deprecated).
18. **public boolean isDaemon():** tests if the thread is a daemon thread.
19. **public void setDaemon(boolean b):** marks the thread as daemon or user thread.
20. **public void interrupt():** interrupts the thread.
21. **public boolean isInterrupted():** tests if the thread has been interrupted.
22. **public static boolean interrupted():** tests if the current thread has been interrupted.

Runnable interface:

The Runnable interface should be implemented by any class whose instances are intended to be executed by a thread. Runnable interface have only one method named run().

1. **public void run():** is used to perform action for a thread.

Starting a thread:

The **start() method** of Thread class is used to start a newly created thread. It performs the following tasks:

- A new thread starts(with new callstack).
- The thread moves from New state to the Runnable state.
- When the thread gets a chance to execute, its target run() method will run.

1) Java Thread Example by extending Thread class

```
class Multi extends Thread{  
public void run(){  
System.out.println("thread is running...");  
}  
public static void main(String args[]){  
Multi t1=new Multi();  
t1.start();  
}
```

```
}  
}
```

2) Java Thread Example by implementing Runnable interface

```
class Multi3 implements Runnable{  
    public void run(){  
        System.out.println("thread is running...");  
    }  
  
    public static void main(String args[]){  
        Multi3 m1=new Multi3();  
        Thread t1 =new Thread(m1); // Using the constructor Thread(Runnable r)  
        t1.start();  
    }  
}
```

Java Applet

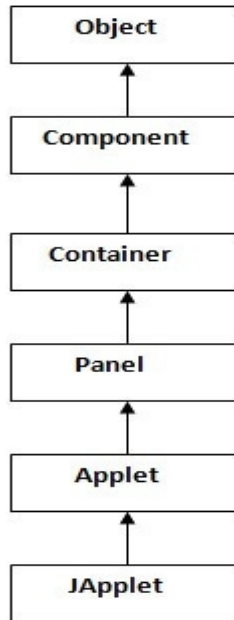
Applet is a special type of program that is embedded in the webpage to generate the dynamic content. It runs inside the browser and works at client side.

Advantage of Applet

There are many advantages of applet. They are as follows:

- It works at client side so less response time.
- Secured
- It can be executed by browsers running under many platforms, including Linux, Windows, Mac Os etc.

Hierarchy of Applet



Lifecycle of Java Applet

1. Applet is initialized.
2. Applet is started.
3. Applet is painted.
4. Applet is stopped.
5. Applet is destroyed.

java.applet.Applet class

For creating any applet `java.applet.Applet` class must be inherited. It provides 4 life cycle methods of applet.

1. `public void init()`: is used to initialize the Applet. It is invoked only once.
2. `public void start()`: is invoked after the `init()` method or browser is maximized. It is used to start the Applet.
3. `public void stop()`: is used to stop the Applet. It is invoked when Applet is stop or browser is minimized.

4. `public void destroy()`: is used to destroy the Applet. It is invoked only once.

java.awt.Component class

The Component class provides 1 life cycle method of applet.

1. `public void paint(Graphics g)`: is used to paint the Applet. It provides Graphics class object that can be used for drawing oval, rectangle, arc etc.

Simple example of Applet by html file:

To execute the applet by html file, create an applet and compile it. After that create an html file and place the applet code in html file. Now click the html file.

```
//First.java
import java.applet.Applet;
import java.awt.Graphics;
public class First extends Applet{

    public void paint(Graphics g){
        g.drawString("welcome",150,150);
    }

}

myapplet.html
<html>
<body>
<applet code="First.class" width="300" height="300">
</applet>
</body>
</html>
```

Displaying Graphics in Applet

java.awt.Graphics class provides many methods for graphics programming.

Commonly used methods of Graphics class:

1. **public abstract void drawString(String str, int x, int y)**: is used to draw the specified string.
2. **public void drawRect(int x, int y, int width, int height)**: draws a rectangle with the specified width and height.

3. **public abstract void fillRect(int x, int y, int width, int height):** is used to fill rectangle with the default color and specified width and height.
4. **public abstract void drawOval(int x, int y, int width, int height):** is used to draw oval with the specified width and height.
5. **public abstract void fillOval(int x, int y, int width, int height):** is used to fill oval with the default color and specified width and height.
6. **public abstract void drawLine(int x1, int y1, int x2, int y2):** is used to draw line between the points(x1, y1) and (x2, y2).
7. **public abstract boolean drawImage(Image img, int x, int y, ImageObserver observer):** is used draw the specified image.
8. **public abstract void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used draw a circular or elliptical arc.
9. **public abstract void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle):** is used to fill a circular or elliptical arc.
10. **public abstract void setColor(Color c):** is used to set the graphics current color to the specified color.
11. **public abstract void setFont(Font font):** is used to set the graphics current font to the specified font.

Example of Graphics in applet:

```
import java.applet.Applet;
import java.awt.*;

public class GraphicsDemo extends Applet{

    public void paint(Graphics g){
        g.setColor(Color.red);
        g.drawString("Welcome",50, 50);
        g.drawLine(20,30,20,300);
        g.drawRect(70,100,30,30);
        g.fillRect(170,100,30,30);
        g.drawOval(70,200,30,30);

        g.setColor(Color.pink);
        g.fillOval(170,200,30,30);
        g.drawArc(90,150,30,30,30,270);
        g.fillArc(270,150,30,30,0,180);
```

```
    }  
    }  
myapplet.html  
    <html>  
    <body>  
    <applet code="GraphicsDemo.class" width="300" height="300">  
    </applet>  
    </body>  
    </html>
```

Displaying Image in Applet

Applet is mostly used in games and animation. For this purpose image is required to be displayed. The `java.awt.Graphics` class provide a method `drawImage()` to display the image.

Syntax of `drawImage()` method:

1. **public abstract boolean drawImage(Image img, int x, int y, ImageObserver observer):** is used draw the specified image.

Example of displaying image in applet:

```
import java.awt.*;
```

```
import java.applet.*;
```

```
public class DisplayImage extends Applet {
```

```
    Image picture;
```

```
    public void init() {  
        picture = getImage(getDocumentBase(),"sonoo.jpg");  
    }
```

```
    public void paint(Graphics g) {  
        g.drawImage(picture, 30,30, this);  
    }
```

```
}
```

In the above example, `drawImage()` method of `Graphics` class is used to display the image. The 4th argument of `drawImage()` method is `ImageObserver` object. The `Component` class implements `ImageObserver` interface. So current class object would also be treated as `ImageObserver` because `Applet` class indirectly extends the `Component` class.

myapplet.html

```
<html>
<body>
<applet code="DisplayImage.class" width="300" height="300">
</applet>
</body>
</html>
```

Animation in Applet

Applet is mostly used in games and animation. For this purpose image is required to be moved.

Example of animation in applet:

```
import java.awt.*;
import java.applet.*;
public class AnimationExample extends Applet {

    Image picture;

    public void init() {
        picture = getImage(getDocumentBase(),"bike_1.gif");
    }

    public void paint(Graphics g) {
        for(int i=0;i<500;i++){
            g.drawImage(picture, i,30, this);

            try {Thread.sleep(100);} catch(Exception e){}
        }
    }
}
```

In the above example, `drawImage()` method of `Graphics` class is used to display the image. The 4th argument of `drawImage()` method is `ImageObserver` object. The `Component` class implements `ImageObserver` interface. So current class object would

also be treated as ImageObserver because Applet class indirectly extends the Component class.

myapplet.html

```
<html>
<body>
<applet code="DisplayImage.class" width="300" height="300">
</applet>
</body>
</html>
```

EventHandling in Applet

As we perform event handling in AWT or Swing, we can perform it in applet also. Let's see the simple example of event handling in applet that prints a message by click on the button.

Example of EventHandling in applet:

```
import java.applet.*;
import java.awt.*;
import java.awt.event.*;
public class EventApplet extends Applet implements ActionListener{
    Button b;
    TextField tf;

    public void init(){
        tf=new TextField();
        tf.setBounds(30,40,150,20);

        b=new Button("Click");
        b.setBounds(80,150,60,50);

        add(b);add(tf);
        b.addActionListener(this);

        setLayout(null);
    }

    public void actionPerformed(ActionEvent e){
        tf.setText("Welcome");
```

```
}  
}
```

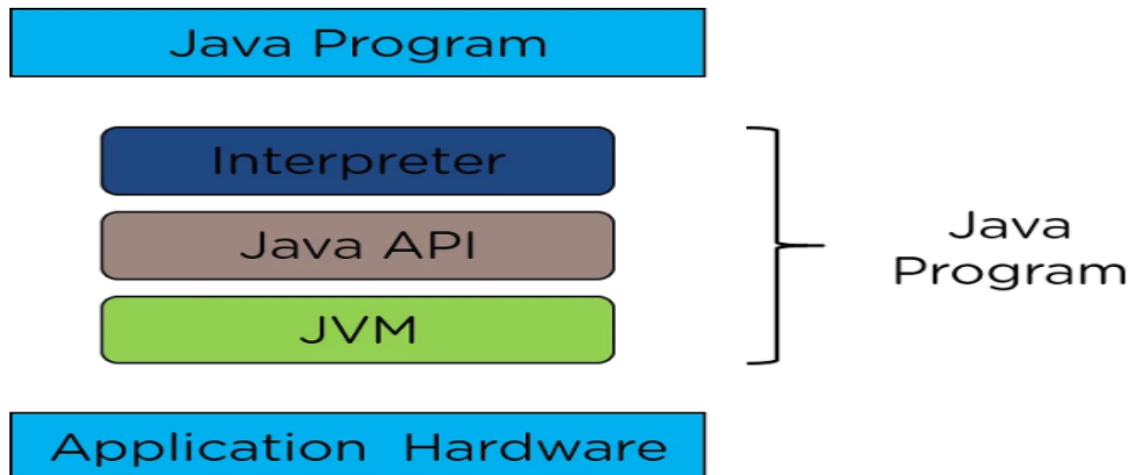
In the above example, we have created all the controls in `init()` method because it is invoked only once when the applet is loaded. The following is the code for `myapplet.html`

```
<html>  
<body>  
<applet code="EventApplet.class" width="300" height="300">  
</applet>  
</body>  
</html>
```

Module IV: Java APIs – overview of APIs, IO Packages, Java Input Stream Classes, Java Output Stream Classes, File Class, Graphic & Sound: AWT and Swing, Graphic methods, Fonts, Loading and Viewing Images, Loading and Playing Sound, AWT & Event Handling, Layouts, **JDBC**.

Java APIs

Java APIs are integrated pieces of software that come with JDKs. APIs in Java provides the interface between two different applications and establish communication.



API: Application Programming Interface

API stands for application program interface. A programmer writing an application program can make a request to the Operating System using API (using graphical user interface or command interface). It is a set of routines, protocols and tools for building software and applications. It may be any type of system like a web-based system, operating-system or a database System. . When you want to use a package/API, you need to use the import keyword and specify which part of the package you want to use. The most common API names start with java, e.g., java.awt.

Java Scanner

Scanner class in Java is found in the java.util package. Java provides various ways to read input from the keyboard, the java.util.Scanner class is one of them.

The Java Scanner class breaks the input into tokens using a delimiter which is whitespace by default. It provides many methods to read and parse various primitive values.

The Java Scanner class is widely used to parse text for strings and primitive types using a regular expression. It is the simplest way to get input in Java. By the help of Scanner in Java, we can get input from the user in primitive types such as int, long, double, byte, float, short, etc.

The Java Scanner class extends Object class and implements Iterator and Closeable interfaces.

The Java Scanner class provides nextXXX() methods to return the type of value such as nextInt(), nextByte(), nextShort(), next(), nextLine(), nextDouble(), nextFloat(), nextBoolean(), etc. To get a single character from the scanner, you can call next().charAt(0) method which returns a single character.

Java Scanner Class Declaration

```
public final class Scanner
    extends Object
    implements Iterator<String>
```

To get the instance of Java Scanner which reads input from the user, we need to pass the input stream (System.in) in the constructor of Scanner class. For Example:

```
Scanner in = new Scanner(System.in);
```

```
Scanner in = new Scanner("Hello Javatpoint");
```

SN	Modifier & Type	Method	Description
1	String	<u>next()</u>	It is used to get the next

			complete token from the scanner which is in use.
2	BigDecimal	<u>nextBigDecimal()</u>	It scans the next token of the input as a BigDecimal.
3	BigInteger	<u>nextBigInteger()</u>	It scans the next token of the input as a BigInteger.
4	boolean	<u>nextBoolean()</u>	It scans the next token of the input into a boolean value and returns that value.
5	byte	<u>nextByte()</u>	It scans the next token of the input as a byte.
6	double	<u>nextDouble()</u>	It scans the next token of the input as a double.
7	float	<u>nextFloat()</u>	It scans the next token of the input as a float.
8	int	<u>nextInt()</u>	It scans the next token of the input as an Int.
9	String	<u>nextLine()</u>	It is used to get the input string that was skipped of the Scanner object.
10	long	<u>nextLong()</u>	It scans the next token of the input as a long.
11	short	<u>nextShort()</u>	It scans the next token of the input as a short.
12	int	<u>radix()</u>	It is used to get the default radix of the Scanner use.

13	void	<u>remove()</u>	It is used when remove operation is not supported by this implementation of Iterator.
14	Scanner	<u>reset()</u>	It is used to reset the Scanner which is in use.
15	Scanner	<u>skip()</u>	It skips input that matches the specified pattern, ignoring delimiters
16	String	<u>toString()</u>	It is used to get the string representation of Scanner using.

Java I/O Tutorial

Java I/O (Input and Output) is used to process the input and produce the output.

Java uses the concept of a stream to make I/O operation fast. The java.io package contains all the classes required for input and output operations.

Stream

A stream is a sequence of data. In Java, a stream is composed of bytes. It's called a stream because it is like a stream of water that continues to flow.

In Java, 3 streams are created for us automatically. All these streams are attached with the console.

- 1) System.out: standard output stream
- 2) System.in: standard input stream
- 3) System.err: standard error stream

Java Output

In Java, you can simply use

System.out.println(); or

System.out.print(); or

System.out.printf();

to send output to standard output (screen).

Here,

System is a class out is a public static field: it accepts output data.

```
class AssignmentOperator {  
    public static void main(String[] args) {  
        System.out.println("Java programming is interesting.");  
    }  
}
```

Output:

Java programming is interesting.

Difference between println(), print() and printf()

print() - It prints string inside the quotes.

println() - It prints string inside the quotes similar like print() method. Then the cursor moves to the beginning of the next line.

printf() - It provides string formatting (similar to printf in C/C++ programming).

Java Input

Java provides different ways to get input from the user. However, in this tutorial, you will learn to get input from user using the object of Scanner class.

In order to use the object of Scanner, we need to import java.util.Scanner package.

```
import java.util.Scanner;
// create an object of Scanner
Scanner input = new Scanner(System.in);

// take input from the user
int number = input.nextInt();

Get float, double and String Input
import java.util.Scanner;

class Input {
    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        // Getting float input
        System.out.print("Enter float: ");
        float myFloat = input.nextFloat();
        System.out.println("Float entered = " + myFloat);

        // Getting double input
        System.out.print("Enter double: ");
        double myDouble = input.nextDouble();
        System.out.println("Double entered = " + myDouble);

        // Getting String input
        System.out.print("Enter text: ");
        String myString = input.next();
        System.out.println("Text entered = " + myString);
    }
}
```

Output:

```
Enter float: 2.343
Float entered = 2.343
Enter double: -23.4
Double entered = -23.4
```

Enter text: Hey!

Text entered = Hey!

Java BufferedReader Class

The `BufferedReader` class of the `java.io` package can be used with other readers to read data (in characters) more efficiently.

Working of BufferedReader

The `BufferedReader` maintains an internal buffer of 8192 characters.

During the read operation in `BufferedReader`, a chunk of characters is read from the disk and stored in the internal buffer. And from the internal buffer characters are read individually.

Hence, the number of communication to the disk is reduced. This is why reading characters is faster using `BufferedReader`.

Create a BufferedReader

In order to create a `BufferedReader`, we must import the `java.io.BufferedReader` package first. Once we import the package, here is how we can create the reader.

```
// Creates a FileReader
```

```
FileReader file = new FileReader(String file);
```

```
// Creates a BufferedReader
```

```
BufferedReader buffer = new BufferedReader(file);
```

In the above example, we have created a `BufferedReader` named `buffer` with the `FileReader` named `file`.

Here, the internal buffer of the `BufferedReader` has the default size of 8192 characters. However, we can specify the size of the internal buffer as well.

```
// Creates a BufferdReader with specified size internal buffer
```

```
BufferedReader buffer = new BufferedReader(file, int size);
```

```
import java.io.FileReader;
```

```
import java.io.BufferedReader;
```

```
class Main {
```

```
public static void main(String[] args) {  
  
    // Creates an array of character  
    char[] array = new char[100];  
  
    try {  
        // Creates a FileReader  
        FileReader file = new FileReader("input.txt");  
  
        // Creates a BufferedReader  
        BufferedReader input = new BufferedReader(file);  
  
        // Reads characters  
        input.read(array);  
        System.out.println("Data in the file: ");  
        System.out.println(array);  
  
        // Closes the reader  
        input.close();  
    }  
  
    catch(Exception e) {  
        e.printStackTrace();  
    }  
}
```

Java AWT Tutorial

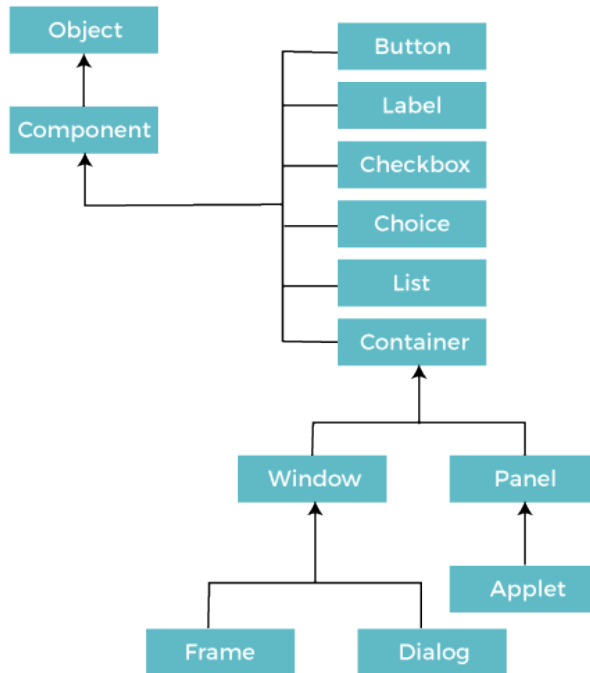
Java AWT (Abstract Window Toolkit) is an API to develop Graphical User Interface (GUI) or windows-based applications in Java.

Java AWT components are platform-dependent i.e. components are displayed according to the view of operating system. AWT is heavy weight i.e. its components are using the resources of underlying operating system (OS).

The java.awt package provides classes for AWT API such as TextField, Label, TextArea, RadioButton, CheckBox, Choice, List etc.

Java AWT Hierarchy

The hierarchy of Java AWT classes are given below.



Components

All the elements like the button, text fields, scroll bars, etc. are called components. In Java AWT, there are classes for each component as shown in above diagram. In order to place every component in a particular position on a screen, we need to add them to a container.

Container

The Container is a component in AWT that can contain another components like buttons, textfields, labels etc. The classes that extends Container class are known as container such as Frame, Dialog and Panel.

It is basically a screen where the where the components are placed at their specific locations. Thus it contains and controls the layout of components.

Types of containers:

There are four types of containers in Java AWT:

1. Window
2. Panel
3. Frame

4. Dialog

Window

The window is the container that have no borders and menu bars. You must use frame, dialog or another window for creating a window. We need to create an instance of Window class to create this container.

Panel

The Panel is the container that doesn't contain title bar, border or menu bar. It is generic container for holding the components. It can have other components like button, text field etc. An instance of Panel class creates a container, in which we can add components.

Frame

The Frame is the container that contain title bar and border and can have menu bars. It can have other components like button, text field, scrollbar etc. Frame is most widely used container while developing an AWT application.

Java AWT Example

To create simple AWT example, you need a frame. There are two ways to create a GUI using Frame in AWT.

1. By extending Frame class (**inheritance**)
2. By creating the object of Frame class (**association**)

AWT Example by Inheritance

```
// importing Java AWT class
import java.awt.*;
```

```
// extending Frame class to our class AWTEExample1
public class AWTEExample1 extends Frame {
```

```
    // initializing using constructor
    AWTEExample1() {
```

```
        // creating a button
```

```
Button b = new Button("Click Me!!");

// setting button position on screen
b.setBounds(30,100,80,30);

// adding button into frame
add(b);

// frame size 300 width and 300 height
setSize(300,300);

// setting the title of Frame
setTitle("This is our basic AWT example");

// no layout manager
setLayout(null);

// now frame will be visible, by default it is not visible
setVisible(true);
}

// main method
public static void main(String args[]) {

// creating instance of Frame class
AWTExample1 f = new AWTExample1();

}
}
```

Event and Listener (Java Event Handling)

Changing the state of an object is known as an event. For example, click on button, dragging mouse etc. The `java.awt.event` package provides many event classes and Listener interfaces for event handling.

Java Event classes and Listener interfaces

Event Classes	Listener Interfaces
ActionEvent	ActionListener
MouseEvent	MouseListener and MouseMotionListener
MouseWheelEvent	MouseWheelListener
KeyEvent	KeyListener
ItemEvent	ItemListener
TextEvent	TextListener
AdjustmentEvent	AdjustmentListener
WindowEvent	WindowListener
ComponentEvent	ComponentListener
ContainerEvent	ContainerListener
FocusEvent	FocusListener

Steps to perform Event Handling

Following steps are required to perform event handling:

1. Register the component with the Listener

Registration Methods

For registering the component with the Listener, many classes provide the registration methods. For example:

- **Button**
 - `public void addActionListener(ActionListener a){}`
- **MenuItem**
 - `public void addActionListener(ActionListener a){}`
- **TextField**
 - `public void addActionListener(ActionListener a){}`
 - `public void addTextListener(TextListener a){}`
- **TextArea**
 - `public void addTextListener(TextListener a){}`
- **Checkbox**
 - `public void addItemListener(ItemListener a){}`

- **Choice**
 - `public void addItemListener(ItemListener a){}`
- **List**
 - `public void addActionListener(ActionListener a){}`
 - `public void addItemListener(ItemListener a){}`

Java Event Handling Code

We can put the event handling code into one of the following places:

1. Within class
2. Other class
3. Anonymous class

Java event handling by implementing ActionListener

```
import java.awt.*;
import java.awt.event.*;
class AEvent extends Frame implements ActionListener{
    TextField tf;
    AEvent(){

//create components
    tf=new TextField();
    tf.setBounds(60,50,170,20);
    Button b=new Button("click me");
    b.setBounds(100,120,80,30);

//register listener
    b.addActionListener(this);//passing current instance

//add components and set size, layout and visibility
    add(b);add(tf);
    setSize(300,300);
    setLayout(null);
    setVisible(true);
    }
    public void actionPerformed(ActionEvent e){
    tf.setText("Welcome");
    }
    public static void main(String args[]){
```

```
new AEvent();  
}  
}
```

Java AWT Button

A button is basically a control component with a label that generates an event when pushed. The **Button** class is used to create a labeled button that has platform independent implementation. The application result in some action when the button is pushed.

When we press a button and release it, AWT sends an instance of **ActionEvent** to that button by calling **processEvent** on the button. The **processEvent** method of the button receives the all the events, then it passes an action event by calling its own method **processActionEvent**. This method passes the action event on to action listeners that are interested in the action events generated by the button.

To perform an action on a button being pressed and released, the **ActionListener** interface needs to be implemented. The registered new listener can receive events from the button by calling **addActionListener** method of the button. The Java application can use the button's action command as a messaging protocol.

AWT Button Class Declaration

1. **public class Button extends Component implements Accessible**

Example:

Sr. no.	Constructor	Description
1.	Button()	It constructs a new button with an empty string i.e. it has no label.
2.	Button (String text)	It constructs a new button with given string as its label.

Button Class Methods

Sr. no.	Method	Description
1.	void setText (String text)	It sets the string message on the button

2.	String getText()	It fetches the String message on the button.
3.	void setLabel (String label)	It sets the label of button with the specified string.
4.	String getLabel()	It fetches the label of the button.
5.	void addNotify()	It creates the peer of the button.
6.	AccessibleContext getAccessibleContext()	It fetched the accessible context associated with the button.
7.	void addActionListener(ActionListener l)	It adds the specified action listener to get the action events from the button.
8.	String getActionCommand()	It returns the command name of the action event fired by the button.
9.	ActionListener[getActionListeners()]	It returns an array of all the action listeners registered on the button.

In the following example, we are handling the button click events by implementing ActionListener Interface.

ButtonExample3.java

```
// importing necessary libraries
import java.awt.*;
import java.awt.event.*;
public class ButtonExample3 {
public static void main(String[] args) {
    // create instance of frame with the label
    Frame f = new Frame("Button Example");
    final TextField tf=new TextField();
```

```
tf.setBounds(50,50, 150,20);
// create instance of button with label
Button b=new Button("Click Here");
// set the position for the button in frame
b.setBounds(50,100,60,30);
b.addActionListener(new ActionListener() {
    public void actionPerformed (ActionEvent e) {
        tf.setText("Welcome to Java programming.");
    }
});
// adding button the frame
f.add(b);
// adding textfield the frame
f.add(tf);
// setting size, layout and visibility
f.setSize(400,400);
f.setLayout(null);
f.setVisible(true);
}
}
```

Java AWT Label

The object of the Label class is a component for placing text in a container. It is used to display a single line of **read only text**. The text can be changed by a programmer but a user cannot edit it directly.

It is called a passive control as it does not create any event when it is accessed. To create a label, we need to create the object of **Label** class.

AWT Label Class Declaration

1. **public class** Label **extends** Component **implements** Accessible

Label class Constructors

Sr. no.	Constructor	Description
1.	Label()	It constructs an empty label.
2.	Label(String text)	It constructs a label with the given string (left justified by default).

3.	Label(String text, int alignment)	It constructs a label with the specified string and the specified alignment.
----	-----------------------------------	--

Label Class Methods

Sr. no.	Method name	Description
1.	void setText(String text)	It sets the texts for label with the specified text.
2.	void setAlignment(int alignment)	It sets the alignment for label with the specified alignment.
3.	String getText()	It gets the text of the label
4.	int getAlignment()	It gets the current alignment of the label.
5.	void addNotify()	It creates the peer for the label.
6.	AccessibleContext getAccessibleContext()	It gets the Accessible Context associated with the label.
7.	protected String paramString()	It returns the string the state of the label.

AWT Label Fields

The java.awt.Component class has following fields:

1. **static int LEFT:** It specifies that the label should be left justified.
2. **static int RIGHT:** It specifies that the label should be right justified.
3. **static int CENTER:** It specifies that the label should be placed in center.

Java AWT Label Example

In the following example, we are creating two labels l1 and l2 using the Label(String text) constructor and adding them into the frame.

LabelExample.java

```
import java.awt.*;
```



```
public class LabelExample {  
    public static void main(String args[]) {  
  
        // creating the object of Frame class and Label class  
        Frame f = new Frame ("Label example");  
        Label l1, l2;  
  
        // initializing the labels  
        l1 = new Label ("First Label.");  
        l2 = new Label ("Second Label.");  
  
        // set the location of label  
        l1.setBounds(50, 100, 100, 30);  
        l2.setBounds(50, 150, 100, 30);  
  
        // adding labels to the frame  
        f.add(l1);  
        f.add(l2);  
  
        // setting size, layout and visibility of frame  
        f.setSize(400,400);  
        f.setLayout(null);  
        f.setVisible(true);  
    }  
}
```

Java AWT TextField

The object of a **TextField** class is a text component that allows a user to enter a single line text and edit it. It inherits **TextComponent** class, which further inherits **Component** class.

When we enter a key in the text field (like key pressed, key released or key typed), the event is sent to **TextField**. Then the **KeyEvent** is passed to the registered **KeyListener**. It can also be done using **ActionEvent**; if the **ActionEvent** is enabled on the text field, then the **ActionEvent** may be fired by pressing return key. The event is handled by the **ActionListener** interface.

AWT TextField Class Declaration

1. **public class** TextField **extends** TextComponent

Sr. no.	Constructor	Description
1.	TextField()	It constructs a new text field component.
2.	TextField(String text)	It constructs a new text field initialized with the given string text to be displayed.
3.	TextField(int columns)	It constructs a new textfield (empty) with given number of columns.
4.	TextField(String text, int columns)	It constructs a new text field with the given text and given number of columns (width).

TextField Class Methods

Sr. no.	Method name	Description
1.	void addNotify()	It creates the peer of text field.
2.	boolean echoCharIsSet()	It tells whether text field has character set for echoing or not.
3.	void addActionListener(ActionListener l)	It adds the specified action listener to receive action events from the text field.
4.	ActionListener[] getActionListeners()	It returns array of all action listeners registered on text field.

Java AWT TextField Example with ActionListener TextFieldExample2.java

```
// importing necessary libraries
import java.awt.*;
import java.awt.event.*;
// Our class extends Frame class and implements ActionListener interface
public class TextFieldExample2 extends Frame implements ActionListener {
    // creating instances of TextField and Button class
    TextField tf1, tf2, tf3;
    Button b1, b2;
```

```
// instantiating using constructor
TextFieldExample2() {
    // instantiating objects of text field and button
    // setting position of components in frame
    tf1 = new TextField();
    tf1.setBounds(50, 50, 150, 20);
    tf2 = new TextField();
    tf2.setBounds(50, 100, 150, 20);
    tf3 = new TextField();
    tf3.setBounds(50, 150, 150, 20);
    tf3.setEditable(false);
    b1 = new Button("+");
    b1.setBounds(50, 200, 50, 50);
    b2 = new Button("-");
    b2.setBounds(120,200,50,50);
    // adding action listener
    b1.addActionListener(this);
    b2.addActionListener(this);
    // adding components to frame
    add(tf1);
    add(tf2);
    add(tf3);
    add(b1);
    add(b2);
    // setting size, layout and visibility of frame
    setSize(300,300);
    setLayout(null);
    setVisible(true);
}
// defining the actionPerformed method to generate an event on buttons
public void actionPerformed(ActionEvent e) {
    String s1 = tf1.getText();
    String s2 = tf2.getText();
    int a = Integer.parseInt(s1);
    int b = Integer.parseInt(s2);
    int c = 0;
    if (e.getSource() == b1){
        c = a + b;
    }
}
```

```
        else if (e.getSource() == b2){
            c = a - b;
        }
        String result = String.valueOf(c);
        tf3.setText(result);
    }
// main method
public static void main(String[] args) {
    new TextFieldExample2();
}
}
```

Java AWT TextArea

The object of a `TextArea` class is a multiline region that displays text. It allows the editing of multiple line text. It inherits `TextComponent` class.

The text area allows us to type as much text as we want. When the text in the text area becomes larger than the viewable area, the scroll bar appears automatically which helps us to scroll the text up and down, or right and left.

AWT `TextArea` Class Declaration

1. **public class** `TextArea` **extends** `TextComponent`

Fields of `TextArea` Class

The fields of `java.awt.TextArea` class are as follows:

- **static int** `SCROLLBARS_BOTH` - It creates and displays both horizontal and vertical scrollbars.
- **static int** `SCROLLBARS_HORIZONTAL_ONLY` - It creates and displays only the horizontal scrollbar.
- **static int** `SCROLLBARS_VERTICAL_ONLY` - It creates and displays only the vertical scrollbar.
- **static int** `SCROLLBARS_NONE` - It doesn't create or display any scrollbar in the text area.

Class constructors:

Sr. no.	Constructor	Description

1.	TextArea()	It constructs a new and empty text area with no text in it.
2.	TextArea (int row, int column)	It constructs a new text area with specified number of rows and columns and empty string as text.
3.	TextArea (String text)	It constructs a new text area and displays the specified text in it.
4.	TextArea (String text, int row, int column)	It constructs a new text area with the specified text in the text area and specified number of rows and columns.
5.	TextArea (String text, int row, int column, int scrollbars)	It constructs a new text area with specified text in text area and specified number of rows and columns and visibility.

Java AWT Checkbox

The Checkbox class is used to create a checkbox. It is used to turn an option on (true) or off (false). Clicking on a Checkbox changes its state from "on" to "off" or from "off" to "on".

AWT Checkbox Class Declaration

1. **public class** Checkbox **extends** Component **implements** ItemSelectable, Accessible

Checkbox Class Constructors

Sr. no.	Constructor	Description
1.	Checkbox()	It constructs a checkbox with no string as the label.
2.	Checkbox(String label)	It constructs a checkbox with the given label.
3.	Checkbox(String label, boolean state)	It constructs a checkbox with the

		given label and sets the given state.
4.	Checkbox(String label, boolean state, CheckboxGroup group)	It constructs a checkbox with the given label, set the given state in the specified checkbox group.
5.	Checkbox(String label, CheckboxGroup group, boolean state)	It constructs a checkbox with the given label, in the given checkbox group and set to the specified state.

Checkbox Class Methods

Sr. no.	Method name	Description
1.	void addItemListener(ItemListener IL)	It adds the given item listener to get the item events from the checkbox.
2.	AccessibleContext getAccessibleContext()	It fetches the accessible context of checkbox.
3.	void addNotify()	It creates the peer of checkbox.
4.	CheckboxGroup getCheckboxGroup()	It determines the group of checkbox.
5.	ItemListener[] getItemListeners()	It returns an array of the item listeners registered on checkbox.
6.	String getLabel()	It fetched the label of checkbox.

Java AWT Choice

The object of Choice class is used to show [popup menu](#) of choices. Choice selected by user is shown on the top of a menu. It inherits Component class.

AWT Choice Class Declaration

1. **public class** Choice **extends** Component **implements** ItemSelectable, Accessible

Choice Class constructor

Sr. no.	Constructor	Description
1.	Choice()	It constructs a new choice menu.

Choice Class Methods

Sr. no.	Method name	Description
1.	void add(String item)	It adds an item to the choice menu.
2.	void addItemListener(ItemListener l)	It adds the item listener that receives item events from the choice menu.
3.	void addNotify()	It creates the peer of choice.
4.	AccessibleContext getAccessibleContext()	It gets the accessible context related to the choice.
5.	String getItem(int index)	It gets the item (string) at the given index position in the choice menu.
6.	int getItemCount()	It returns the number of items of the choice menu.

Java AWT Choice Example with ActionListener

In the following example, we are creating a choice menu with 5 items. Along with that we are creating a button and a label. Here, we are adding an event to the button component using `addActionListener(ActionListener a)` method i.e. the selected item from the choice menu is displayed on the label when the button is clicked.

ChoiceExample2.java

```
// importing necessary packages
import java.awt.*;
import java.awt.event.*;

public class ChoiceExample2 {
```

```
// class constructor
ChoiceExample2() {

    // creating a frame
    Frame f = new Frame();

    // creating a final object of Label class
    final Label label = new Label();

    // setting alignment and size of label component
    label.setAlignment(Label.CENTER);
    label.setSize(400, 100);

    // creating a button
    Button b = new Button("Show");

    // setting the bounds of button
    b.setBounds(200, 100, 50, 20);

    // creating final object of Choice class
    final Choice c = new Choice();

    // setting bounds of choice menu
    c.setBounds(100, 100, 75, 75);

    // adding 5 items to choice menu
    c.add("C");
    c.add("C++");
    c.add("Java");
    c.add("PHP");
    c.add("Android");

    // adding above components into the frame
    f.add(c);
    f.add(label);
    f.add(b);

    // setting size, layout and visibility of frame
```



```
f.setSize(400, 400);
f.setLayout(null);
f.setVisible(true);

// adding event to the button
// which displays the selected item from the list when button is clicked
b.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String data = "Programming language Selected: "+
c.getItem(c.getSelectedIndex());
        label.setText(data);
    }
});
}
```

```
// main method
public static void main(String args[])
{
    new ChoiceExample2();
}
}
```

Java AWT List

The object of List class represents a list of text items. With the help of the List class, user can choose either one item or multiple items. It inherits the Component class.

AWT List class Declaration

1. **public class** List **extends** Component **implements** ItemSelectable, Accessible

AWT List Class Constructors

Sr. no.	Constructor	Description
1.	List()	It constructs a new scrolling list.
2.	List(int row_num)	It constructs a new scrolling list initialized with the given number of rows visible.

3.	List(int row_num, Boolean multipleMode)	It constructs a new scrolling list initialized which displays the given number of rows.
----	---	---

List Class Methods

Sr. no.	Method name	Description
1.	void add(String item)	It adds the specified item into the end of scrolling list.
2.	void add(String item, int index)	It adds the specified item into list at the given index position.
3.	void addActionListener(ActionListener l)	It adds the specified action listener to receive action events from list.
4.	void addItemListener(ItemListener l)	It adds specified item listener to receive item events from list.
5.	void addNotify()	It creates peer of list.
6.	void deselect(int index)	It deselects the item at given index position.
7.	AccessibleContext getAccessibleContext()	It fetches the accessible context related to the list.
8.	ActionListener[] getActionListeners()	It returns an array of action listeners registered on the list.

Java AWT Scrollbar

The object of Scrollbar class is used to add horizontal and vertical scrollbar. Scrollbar is a GUI component allows us to see invisible number of rows and columns.

It can be added to top-level container like Frame or a component like Panel. The Scrollbar class extends the Component class.

AWT Scrollbar Class Declaration

public class Scrollbar extends Component implements Adjustable, Accessible

Scrollbar Class Fields

The fields of java.awt.Image class are as follows:

- **static int HORIZONTAL** - It is a constant to indicate a horizontal scroll bar.
- **static int VERTICAL** - It is a constant to indicate a vertical scroll bar.

Scrollbar Class Constructors

Sr. no.	Constructor	Description
1	Scrollbar()	Constructs a new vertical scroll bar.
2	Scrollbar(int orientation)	Constructs a new scroll bar with the specified orientation.
3	Scrollbar(int orientation, int value, int visible, int minimum, int maximum)	Constructs a new scroll bar with the specified orientation, initial value, visible amount, and minimum and maximum values.

Scrollbar Class Methods

Sr. no.	Method name	Description
1.	void addAdjustmentListener (AdjustmentListener l)	It adds the given adjustment listener to receive instances of AdjustmentEvent from the scroll bar.
2.	void addNotify()	It creates the peer of scroll bar.
3.	int getBlockIncrement()	It gets the block increment of the scroll bar.
4.	int getMaximum()	It gets the maximum value of the scroll bar.
5.	int getMinimum()	It gets the minimum value of the scroll bar.
6.	int getOrientation()	It returns the orientation of scroll bar.
7.	int getUnitIncrement()	It fetches the unit increment of the scroll bar.

8.	<code>int getValue()</code>	It fetches the current value of scroll bar.
----	-----------------------------	---

Java ActionListener Interface

The Java ActionListener is notified whenever you click on the button or menu item. It is notified against ActionEvent. The ActionListener interface is found in java.awt.event package. It has only one method: actionPerformed().

actionPerformed() method

The actionPerformed() method is invoked automatically whenever you click on the registered component.

1. **public abstract void** actionPerformed(ActionEvent e);

to write ActionListener

The common approach is to implement the ActionListener. If you implement the ActionListener class, you need to follow 3 steps:

- 1) Implement the ActionListener interface in the class:

```
public class ActionListenerExample Implements ActionListener
```

- 2) Register the component with the Listener:

```
component.addActionListener(instanceOfListenerclass);
```

- 3) Override the actionPerformed() method:

```
public void actionPerformed(ActionEvent e){  
    //Write the code here  
}
```

Java MouseListener Interface

The Java MouseListener is notified whenever you change the state of mouse. It is notified against MouseEvent. The MouseListener interface is found in java.awt.event package. It has five methods.

Methods of MouseListener interface

The signature of 5 methods found in MouseListener interface are given below:

1. **public abstract void** mouseClicked(MouseEvent e);
2. **public abstract void** mouseEntered(MouseEvent e);
3. **public abstract void** mouseExited(MouseEvent e);

4. **public abstract void** mousePressed(MouseEvent e);
5. **public abstract void** mouseReleased(MouseEvent e);

```
import java.awt.*;
import java.awt.event.*;
public class MouseListenerExample2 extends Frame implements MouseListener{
    MouseListenerExample2(){
        addMouseListener(this);

        setSize(300,300);
        setLayout(null);
        setVisible(true);
    }
    public void mouseClicked(MouseEvent e) {
        Graphics g=getGraphics();
        g.setColor(Color.BLUE);
        g.fillOval(e.getX(),e.getY(),30,30);
    }
    public void mouseEntered(MouseEvent e) {}
    public void mouseExited(MouseEvent e) {}
    public void mousePressed(MouseEvent e) {}
    public void mouseReleased(MouseEvent e) {}

    public static void main(String[] args) {
        new MouseListenerExample2();
    }
}
```

Java Swing Tutorial

Java Swing tutorial is a part of Java Foundation Classes (JFC) that is *used to create window-based applications*. It is built on the top of AWT (Abstract Windowing Toolkit) API and entirely written in java.

Unlike AWT, Java Swing provides platform-independent and lightweight components.

The javax.swing package provides classes for java swing API such as JButton, JTextField, JTextArea, JRadioButton, JCheckbox, JMenu, JColorChooser etc.

Difference between AWT and Swing

There are many differences between java awt and swing that are given below.

No.	Java AWT	Java Swing
1)	AWT components are platform-dependent .	Java swing components are platform-independent .
2)	AWT components are heavyweight .	Swing components are lightweight .
3)	AWT doesn't support pluggable look and feel .	Swing supports pluggable look and feel .
4)	AWT provides less components than Swing.	Swing provides more powerful components such as tables, lists, scrollpanes, colorchooser, tabbedpane etc.
5)	AWT doesn't follows MVC (Model View Controller) where model represents data, view represents presentation and controller acts as an interface between model and view.	Swin

Java Swing Examples

There are two ways to create a frame:

By creating the object of Frame class (association)

By extending Frame class (inheritance)

We can write the code of swing inside the main(), constructor or any other method.

Simple Java Swing Example

a simple swing example where we are creating one button and adding it on the JFrame object inside the main() method.

File: FirstSwingExample.java

```
import javax.swing.*;

public class FirstSwingExample {
    public static void main(String[] args) {
        JFrame f=new JFrame();//creating instance of JFrame
```

```
JButton b=new JButton("click");//creating instance of JButton
b.setBounds(130,100,100, 40);//x axis, y axis, width, height

f.add(b);//adding button in JFrame

f.setSize(400,500);//400 width and 500 height
f.setLayout(null);//using no layout managers
f.setVisible(true);//making the frame visible
}
}
```

Java JButton

The JButton class is used to create a labeled button that has platform independent implementation. The application result in some action when the button is pushed. It inherits AbstractButton class.

JButton class declaration

the declaration for javax.swing.JButton class.

1. **public class JButton extends AbstractButton implements Accessible**

Commonly used Constructors:

Constructor	Description
JButton()	It creates a button with no text and icon.
JButton(String s)	It creates a button with the specified text.
JButton(Icon i)	It creates a button with the specified icon object.

Commonly used Methods of AbstractButton class:

Methods	Description
void setText(String s)	It is used to set specified text on button
String getText()	It is used to return the text of the button.
void setEnabled(boolean b)	It is used to enable or disable the button.
void setIcon(Icon b)	It is used to set the specified Icon on the

	button.
Icon getIcon()	It is used to get the Icon of the button.
void setMnemonic(int a)	It is used to set the mnemonic on the button.
void addActionListener(ActionListener a)	It is used to add the action listener to this object.

Java JLabel

The object of JLabel class is a component for placing text in a container. It is used to display a single line of read only text. The text can be changed by an application but a user cannot edit it directly. It inherits JComponent class.

JLabel class declaration

Let's see the declaration for javax.swing.JLabel class.

1. **public class JLabel extends JComponent implements** SwingConstants, Accessibl
e

Commonly used Constructors:

Constructor	Description
JLabel()	Creates a JLabel instance with no image and with an empty string for the title.
JLabel(String s)	Creates a JLabel instance with the specified text.
JLabel(Icon i)	Creates a JLabel instance with the specified image.
JLabel(String s, Icon i, int horizontalAlignment)	Creates a JLabel instance with the specified text, image, and horizontal alignment.

Commonly used Methods:

Methods	Description
---------	-------------

String getText()	It returns the text string that a label displays.
void setText(String text)	It defines the single line of text this component will display.
void setHorizontalAlignment(int alignment)	It sets the alignment of the label's contents along the X axis.
Icon getIcon()	It returns the graphic image that the label displays.
int getHorizontalAlignment()	It returns the alignment of the label's contents along the X axis.

Java JTextField

The object of a JTextField class is a text component that allows the editing of a single line text. It inherits JTextComponent class.

JTextField class declaration

Let's see the declaration for javax.swing.JTextField class.

1. **public class JTextField extends JTextComponent implements SwingConstants**

Commonly used Constructors:

Constructor	Description
JTextField()	Creates a new TextField
JTextField(String text)	Creates a new TextField initialized with the specified text.
JTextField(String text, int columns)	Creates a new TextField initialized with the specified text and columns.
JTextField(int columns)	Creates a new empty TextField with the specified number of columns.

Commonly used Methods:

Methods	Description
<code>void addActionListener(ActionListener l)</code>	It is used to add the specified action listener to receive action events from this textfield.
<code>Action getAction()</code>	It returns the currently set Action for this ActionEvent source, or null if no Action is set.
<code>void setFont(Font f)</code>	It is used to set the current font.
<code>void removeActionListener(ActionListener l)</code>	It is used to remove the specified action listener so that it no longer receives action events from this textfield.

Java JTextField Example with ActionListener

```
import javax.swing.*;
import java.awt.event.*;
public class TextFieldExample implements ActionListener{
    JTextField tf1,tf2,tf3;
    JButton b1,b2;
    TextFieldExample(){
        JFrame f= new JFrame();
        tf1=new JTextField();
        tf1.setBounds(50,50,150,20);
        tf2=new JTextField();
        tf2.setBounds(50,100,150,20);
        tf3=new JTextField();
        tf3.setBounds(50,150,150,20);
        tf3.setEditable(false);
        b1=new JButton("+");
        b1.setBounds(50,200,50,50);
        b2=new JButton("-");
        b2.setBounds(120,200,50,50);
        b1.addActionListener(this);
```

```
b2.addActionListener(this);
f.add(tf1);f.add(tf2);f.add(tf3);f.add(b1);f.add(b2);
f.setSize(300,300);
f.setLayout(null);
f.setVisible(true);
}
public void actionPerformed(ActionEvent e) {
    String s1=tf1.getText();
    String s2=tf2.getText();
    int a=Integer.parseInt(s1);
    int b=Integer.parseInt(s2);
    int c=0;
    if(e.getSource()==b1){
        c=a+b;
    }else if(e.getSource()==b2){
        c=a-b;
    }
    String result=String.valueOf(c);
    tf3.setText(result);
}
public static void main(String[] args) {
    new TextFieldExample();
} }
```

Java JTextArea

The object of a JTextArea class is a multi line region that displays text. It allows the editing of multiple line text. It inherits JTextComponent class

JTextArea class declaration

Let's see the declaration for javax.swing.JTextArea class.

1. **public class** JTextArea **extends** JTextComponent

Commonly used Constructors:

Constructor	Description
JTextArea()	Creates a text area that displays no text initially.
JTextArea(String s)	Creates a text area that displays specified text

	initially.
<code>JTextArea(int row, int column)</code>	Creates a text area with the specified number of rows and columns that displays no text initially.
<code>JTextArea(String s, int row, int column)</code>	Creates a text area with the specified number of rows and columns that displays specified text.

Commonly used Methods:

Methods	Description
<code>void setRows(int rows)</code>	It is used to set specified number of rows.
<code>void setColumns(int cols)</code>	It is used to set specified number of columns.
<code>void setFont(Font f)</code>	It is used to set the specified font.
<code>void insert(String s, int position)</code>	It is used to insert the specified text on the specified position.
<code>void append(String s)</code>	It is used to append the given text to the end of the document.

Java JPasswordField

The object of a JPasswordField class is a text component specialized for password entry. It allows the editing of a single line of text. It inherits JTextField class.

JPasswordField class declaration

the declaration for `javax.swing.JPasswordField` class.

1. **public class JPasswordField extends JTextField**

Commonly used Constructors:

Constructor	Description
JPasswordField()	Constructs a new JPasswordField, with a default document, null starting text string, and 0 column width.
JPasswordField(int columns)	Constructs a new empty JPasswordField with the specified number of columns.
JPasswordField(String text)	Constructs a new JPasswordField initialized with the specified text.
JPasswordField(String text, int columns)	Construct a new JPasswordField initialized with the specified text and columns.

Java JCheckBox

The JCheckBox class is used to create a checkbox. It is used to turn an option on (true) or off (false). Clicking on a CheckBox changes its state from "on" to "off" or from "off" to "on ".It inherits JToggleButton class.

JCheckBox class declaration

Let's see the declaration for javax.swing.JCheckBox class.

1. **public class JCheckBox extends JToggleButton implements Accessible**

Commonly used Constructors:

Constructor	Description
JCheckBox()	Creates an initially unselected check box button with no text, no icon.
JChechBox(String s)	Creates an initially unselected check box with text.
JCheckBox(String text, boolean selected)	Creates a check box with text and specifies whether or not it is initially selected.

JCheckBox(Action a)	Creates a check box where properties are taken from the Action supplied.
---------------------	--

Java JDBC

JDBC stands for Java Database Connectivity. JDBC is a Java API to connect and execute the query with the database. It is a part of JavaSE (Java Standard Edition). JDBC API uses JDBC drivers to connect with the database. There are four types of JDBC drivers:

- JDBC-ODBC Bridge Driver,
- Native Driver,
- Network Protocol Driver, and
- Thin Driver

Use JDBC

Before JDBC, ODBC API was the database API to connect and execute the query with the database. But, ODBC API uses ODBC driver which is written in C language (i.e. platform dependent and unsecured). That is why Java has defined its own API (JDBC API) that uses JDBC drivers (written in Java language).

We can use JDBC API to handle database using Java program and can perform the following activities:

1. Connect to the database
2. Execute queries and update statements to the database
3. Retrieve the result received from the database.

API

API (Application programming interface) is a document that contains a description of all the features of a product or software. It represents classes and interfaces that software programs can follow to communicate with each other. An API can be created for applications, libraries, operating systems, etc.

JDBC Driver

JDBC Driver is a software component that enables java application to interact with the database. There are 4 types of JDBC drivers:

1. JDBC-ODBC bridge driver
2. Native-API driver (partially java driver)
3. Network Protocol driver (fully java driver)

4. Thin driver (fully java driver)

1) JDBC-ODBC bridge driver

The JDBC-ODBC bridge driver uses ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls. This is now discouraged because of thin driver.

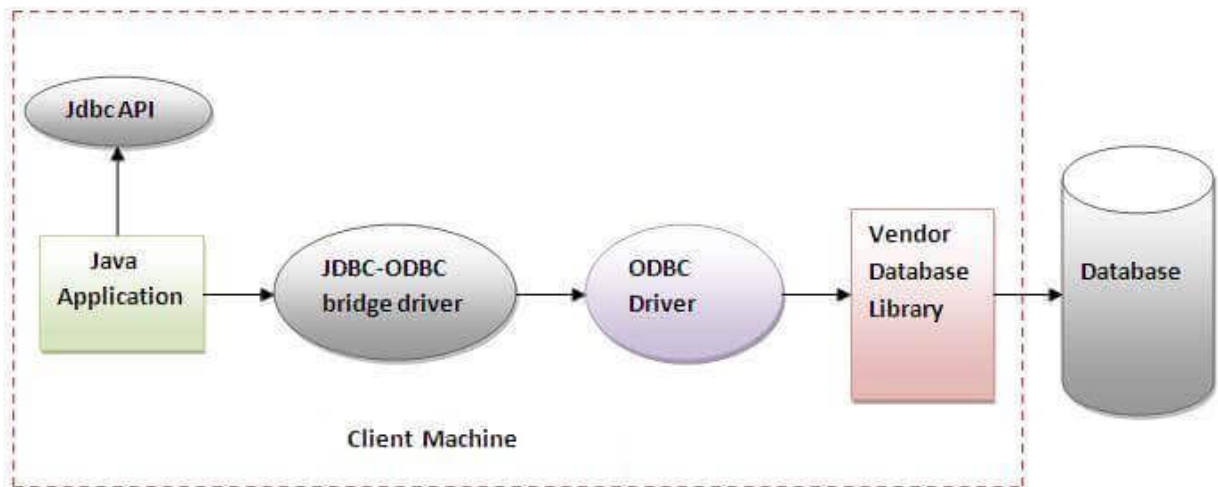


Figure- JDBC-ODBC Bridge Driver

Oracle does not support the JDBC-ODBC Bridge from Java 8. Oracle recommends that you use JDBC drivers provided by the vendor of your database instead of the JDBC-ODBC Bridge.

Advantages:

- easy to use.
- can be easily connected to any database.

Disadvantages:

- Performance degraded because JDBC method call is converted into the ODBC function calls.
- The ODBC driver needs to be installed on the client machine.

2) Native-API driver

The Native API driver uses the client-side libraries of the database. The driver converts JDBC method calls into native calls of the database API. It is not written entirely in java.

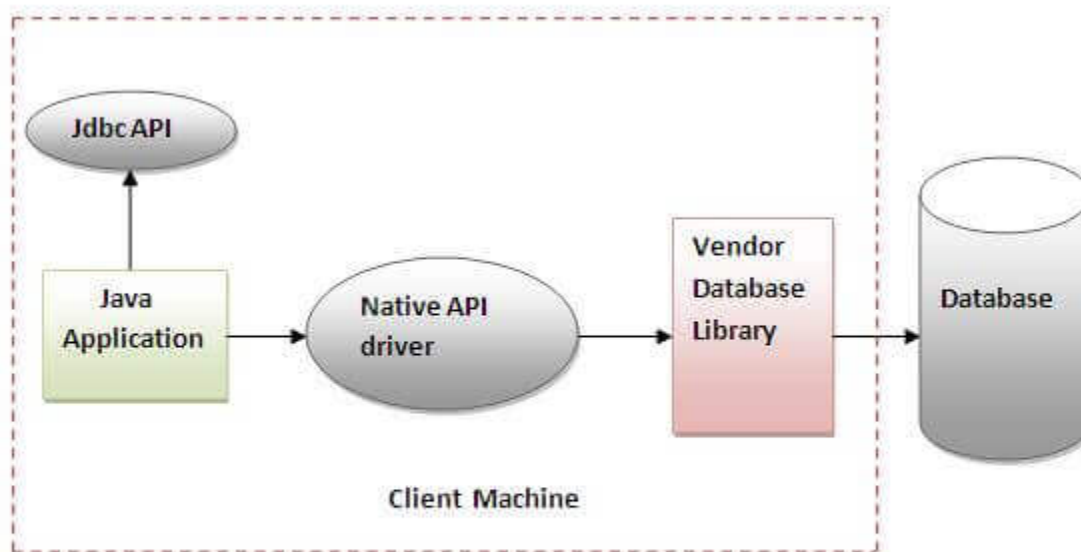


Figure- Native API Driver

Advantage:

- performance upgraded than JDBC-ODBC bridge driver.

Disadvantage:

- The Native driver needs to be installed on the each client machine.
- The Vendor client library needs to be installed on client machine.

3) Network Protocol driver

The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. It is fully written in java.

Advantage:

- No client side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.

Disadvantages:

- Network support is required on client machine.
- Requires database-specific coding to be done in the middle tier.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

4) Thin driver

The thin driver converts JDBC calls directly into the vendor-specific database protocol. That is why it is known as thin driver. It is fully written in Java language.

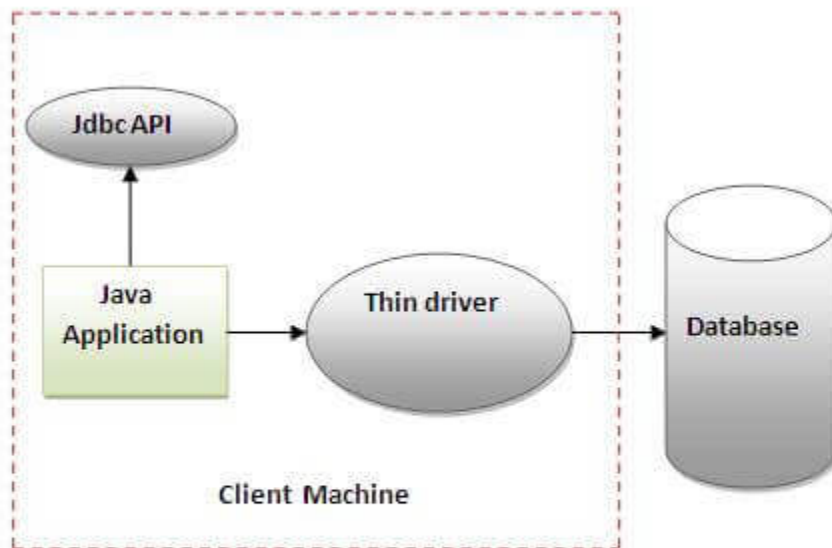


Figure- Thin Driver

Advantage:

- Better performance than all other drivers.
- No software is required at client side or server side.

Disadvantage:

- Drivers depend on the Database.

Java Database Connectivity with 5 Steps

There are 5 steps to connect any java application with the database using JDBC. These steps are as follows:

- Register the Driver class
- Create connection
- Create statement
- Execute queries
- Close connection

1) Register the driver class

The **forName()** method of Class class is used to register the driver class. This method is used to dynamically load the driver class.

Syntax of forName() method

1. **public static void** forName(String className)**throws** ClassNotFoundException

Example to register the OracleDriver class

Here, Java program is loading oracle driver to establish database connection.

1. Class.forName("oracle.jdbc.driver.OracleDriver");

2) Create the connection object

The **getConnection()** method of DriverManager class is used to establish connection with the database.

Syntax of getConnection() method

- 1) **public static** Connection getConnection(String url)**throws** SQLException
- 2) **public static** Connection getConnection(String url,String name,String password)

throws SQLException

Example to establish connection with the Oracle database

1. Connection con=DriverManager.getConnection(
2. "jdbc:oracle:thin:@localhost:1521:xe","system","password");

3) Create the Statement object

The createStatement() method of Connection interface is used to create statement. The object of statement is responsible to execute queries with the database.

Syntax of createStatement() method

1. **public** Statement createStatement()**throws** SQLException

Example to create the statement object

1. Statement stmt=con.createStatement();

4) Execute the query

The executeQuery() method of Statement interface is used to execute queries to the database. This method returns the object of ResultSet that can be used to get all the records of a table.

Syntax of executeQuery() method

1. **public** ResultSet executeQuery(String sql)**throws** SQLException

Example to execute query

```
ResultSet rs=stmt.executeQuery("select * from emp");

while(rs.next()){
    System.out.println(rs.getInt(1)+" "+rs.getString(2));
}
```

5) Close the connection object

By closing connection object statement and ResultSet will be closed automatically. The close() method of Connection interface is used to close the connection.

Syntax of close() method

1. **public void close()throws SQLException**

Example to close connection

1. con.close();

Java Database Connectivity with Oracle

To connect java application with the oracle database, we need to follow 5 following steps. In this example, we are using Oracle 10g as the database. So we need to know following information for the oracle database:

1. **Driver class:** The driver class for the oracle database is **oracle.jdbc.driver.OracleDriver**.
2. **Connection URL:** The connection URL for the oracle10G database is **jdbc:oracle:thin:@localhost:1521:xe** where jdbc is the API, oracle is the database, thin is the driver, localhost is the server name on which oracle is running, we may also use IP address, 1521 is the port number and XE is the Oracle service name. You may get all these information from the tnsnames.ora file.
3. **Username:** The default username for the oracle database is **system**.
4. **Password:** It is the password given by the user at the time of installing the oracle database.

Java Database Connectivity with MySQL

To connect Java application with the MySQL database, we need to follow 5 following steps.

In this example we are using MySql as the database. So we need to know following informations for the mysql database:

1. **Driver class:** The driver class for the mysql database is **com.mysql.jdbc.Driver**.
2. **Connection URL:** The connection URL for the mysql database is **jdbc:mysql://localhost:3306/sonoo** where jdbc is the API, mysql is the database, localhost is the server name on which mysql is running, we may also use IP address, 3306 is the port number and sonoo is the database name. We may use any database, in such case, we need to replace the sonoo with our database name.
3. **Username:** The default username for the mysql database is **root**.
4. **Password:** It is the password given by the user at the time of installing the mysql database. In this example, we are going to use root as the password.